Knowledge and Information Systems (2025) 67:2967–3022
https://doi.org/10.1007/s10115-024-02310-4

REGULAR PAPER

Large language models: a survey of their development, capabilities, and applications

Yadagiri Annepaka[1] · Partha Pakray[1]

Received: 12 September 2024 / Revised: 2 December 2024 / Accepted: 16 December 2024 /
Published online: 31 December 2024
© The Author(s), under exclusive licence to Springer-Verlag London Ltd., part of Springer Nature 2024

Abstract
Large language models can generate text, respond to queries, and translate between languages, as recent research demonstrates. As a new and essential part of computational linguistics, LLMs can understand complex speech patterns and respond appropriately and rationally in the given context. Research contributions have significantly increased as a result of LLMs growing popularity. Nevertheless, the rate of increase has been so quick that evaluating the cumulative impact of these developments is challenging. Keeping abreast with all the LLM research that has surfaced in such a short time and gaining a thorough understanding of the present state of the field's study has become challenging. The scientific community would gain from a succinct but thorough LLM summary covering their history, architecture, applications, issues, influence, and resources. This paper covers many model types while reviewing LLMs' fundamental principles and notions. It summarizes earlier research, the evolution of LLMs throughout time, transformer history, and the range of tools and training methods applied to these models. The study also looks at the many fields in which LLMs are used, including business, social work, education, healthcare, and agriculture. Furthermore, it illustrates how LLMs impact society, mold AI's future, and find valuable uses in problem-solving. This review article aims to provide a comprehensive overview of the history of LLMs, pre-trained architectures, applications, problems, and future directions for practitioners, researchers, and experts.

Keywords Natural language processing · Large language models · Generative AI · Neural networks

1 Introduction

Natural Language Processing (NLP) [1] is a subfield of Artificial Intelligence (AI) [2] that focuses on enabling computer systems to understand and generate human languages, and it has made significant contributions to advancements in this field. It involves linguistic com-

✉ Partha Pakray
partha@cse.nits.ac.in

Yadagiri Annepaka
annepaka22_rs@cse.nits.ac.in

1 Department of Computer Science & Engineering, National Institute of Technology Silchar, Assam 788010, India

putational and Machine-Learning (ML) methods to process the text and speech data [3]. It is critical to thoroughly and efficiently analyze text and speech data and work through the differences in dialect slang and grammatical irregularities typical in day-to-day conversation. Speech and languages are extraordinary instruments for human expression and communication, unfolding from early childhood and persisting throughout a lifetime [4, 5]. However, machines need the aid of advanced AI to comprehend and converse in human language. Thus, an enduring scientific pursuit has been to realize machine proficiency comparable to human reading, writing, and communication skills [6]. A language model typically represents the probability distribution of a natural language. The first significant Statistical Language Model (SLM) was introduced in 1980 and 1990s. Over the next decade, IBM carried out "*Shannon-style*" experiments to observe and analyze human performance in text prediction and correction, aiming to identify potential improvements for language modeling, as discussed [7].

Language Models (*LMs*) are helpful for many tasks, including Machine Translation (MT) [8], Natural Language Generation (*NLG*) (*for producing more human-like text*), optical character recognition, and handwriting recognition [9]. They also play a role in grammar induction [10], Information Retrieval (*IR*) [11], and speech recognition [12]. These models help to avoid generating low-probability sequences, thereby reducing nonsensical outputs. LM focuses on comprehending a given language's word probability distribution so that the model can generate a new text. According to Fig. 1 and as discussed in [13], language models can be categorized into transformer-based, statistical, machine-learning, and deep-learning approaches. They also finish sentences and forecast the likelihood of different word combinations. Early LMs, like n-gram models [14], used simple statistical methods and word sequence frequency counts to estimate probabilities. Integrating Deep Learning (DL) into NLP and the availability of extensive public datasets and powerful computing equipment for processing large-scale data using complex methods has facilitated the construction of comprehensive LMs.

However, LMs encounter several challenges, including managing rare or previously unseen words, the risk of overfitting during training, and the difficulty in capturing complex linguistic phenomena. To overcome these challenges, researchers have been actively improving LM architectures and refining training methodologies, as highlighted in [15]. A large language model (*LLM*) is distinguished by its capacity to perform various NLP tasks, including text generation and classification. These models gain their capabilities through a rigorous training process that involves learning statistical relationships from extensive text corpora, employing both self-supervised and semi-supervised methodologies as described by [16]. As illustrated in Fig. 2, LLMs can facilitate text generation, a subset of generative AI, by iteratively predicting the next token or word based on the provided input text, as discussed by [17].

LLMs are an Artificial Neural Network (*ANN*) described by [18, 19]. The most potent and effective LLMs are typically constructed using a transformer-based, decoder-only architecture. However, some recent implementations have utilized alternative architectures, including variants of Recurrent Neural Networks (*RNNs*) [20, 21] and state space models like Mamba [22]. Before 2020, fine-tuning was the primary method for adapting models to perform specific tasks. However, with larger models such as GPT (*Generative pre-trained transformer*)-3, prompt engineering has emerged as an alternative approach, enabling these models to achieve similar results without requiring extensive fine-tuning [19]. Along with learning about the biases and mistakes included in the corpora, users should also learn about the syntactic, semantic, and "ontological" aspects of human language corpora by [23].

LLMs, sometimes referred to as '*transformative*' [24] or '*next-generation*' [25], represent a substantial leap forward in NLP. These models utilize DL methods, especially transformer

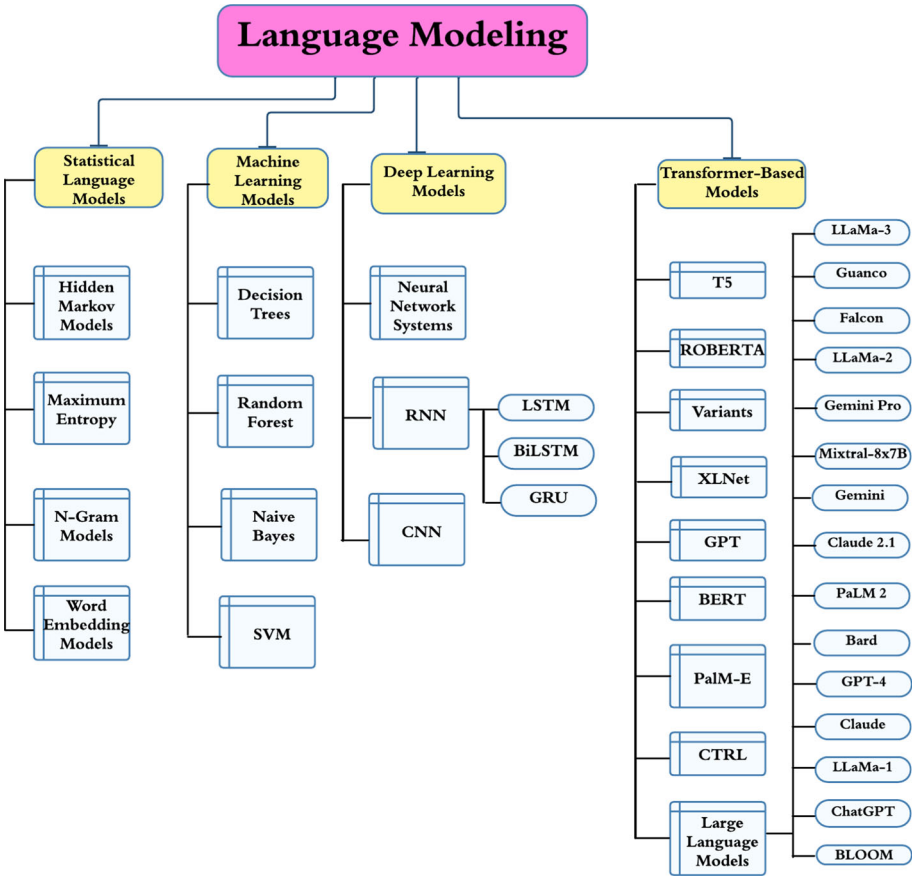

Fig. 1 Types of language models [30]

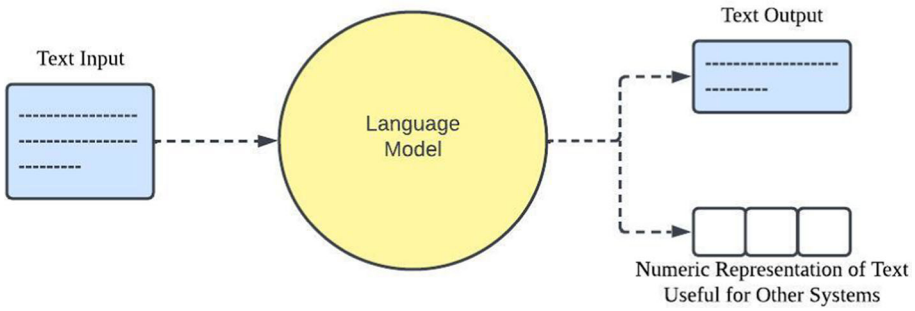

Fig. 2 Workflow of a language model

architectures, to discern intricate patterns and structures within language data, as noted by [26]. A defining feature of LLMs is their capability to handle extensive datasets, including unstructured text, and to understand the semantic connections between words and phrases. Furthermore, these models can process various data modalities, including visual, audio, and

audiovisual information, as well as multi-modal data, learning the semantic connections within them [27, 28]. This capability has dramatically improved machine understanding and generation of human-like language, as highlighted by [29]. Figure 2 illustrates the basic LLM architecture workflow. The process begins with the LLM receiving text data from various sources. This text data is then forwarded to the subsequent stage, where the LM operates. The model undergoes training during this stage by generating text outputs and numerical representations of pertinent information. This involves processing numerical data inputs and integrating with other systems as necessary.

2 History of large language models

Before 2017, only a few LMs could be considered significant compared to the computational capabilities of the time. In the 1990s, IBM's alignment models were pivotal in developing statistical language modeling. A smoothed n-gram model trained on 0.3 billion words in 2001 set a new benchmark for perplexity in that era [31]. With the rise of the Internet in the 2000s, researchers began to create large-scale datasets, often called '*web corpora*' [32], which were used to train SLMs [33]. By 2009, statistical LMs had primarily surpassed symbolic models in most NLP tasks due to their ability to effectively process vast amounts of data [34]. Following the rise of neural networks in image processing around 2012 [35], these models were also adapted for language modeling. In 2016, Google transitioned its translation service to Neural Machine Translation (*NMT*). Before the advent of transformers, this transition was achieved using sequence-to-sequence (*seq2seq*) deep LSTM networks.

During the 2017 NeurIPS conference, Google researchers presented the transformer architecture in their groundbreaking paper, "*Attention Is All You Need*." This work aimed to enhance the seq2seq technology [18] introduced in 2014, primarily building on the attention mechanism developed by [36] in 2014. In 2018, BERT was introduced and rapidly became widely adopted. While the original transformer model includes encoder and decoder blocks, BERT utilizes only the encoder component. The brief history of LMs is shown in Fig. 3. While GPT-1, a decoder-only model, was introduced in 2018, GPT-2, released in 2019, garnered significant attention. Initially, OpenAI withheld GPT-2 due to concerns over its potential for malicious use [37]. GPT-3, released in 2020, advanced the model further and, as of 2024, is only accessible via API, with no option to download and run it locally. However, the 2022 launch of the ChatGPT browser-based interface captured the public imagination, generating considerable media interest. GPT-4, launched in 2023, was lauded for its improved accuracy and multimodal capabilities [38], which some call the "*holy grail*." OpenAI has not disclosed the architecture or the parameter count for GPT-4.

Most competing LMs have focused on matching or exceeding the GPT series, particularly regarding parameter size. Since 2022, open-source models have gained traction, beginning with BLOOM and LLaMA, though both have usage restrictions. Mistral AI has released more permissive models under the Apache License, such as Mistral 7B and Mixtral 8x7B. As of June 2024, the Instruction-fine-tuned version of the 70-billion-parameter LLaMA 3 model is regarded as the most powerful open LLM, surpassing GPT-3.5 in performance, though still not as powerful as GPT-4, according to the LMSYS Chatbot Arena Leaderboard. By 2024, the largest and most capable models are primarily based on the Transformer architecture, although recent developments have also explored alternative architectures.

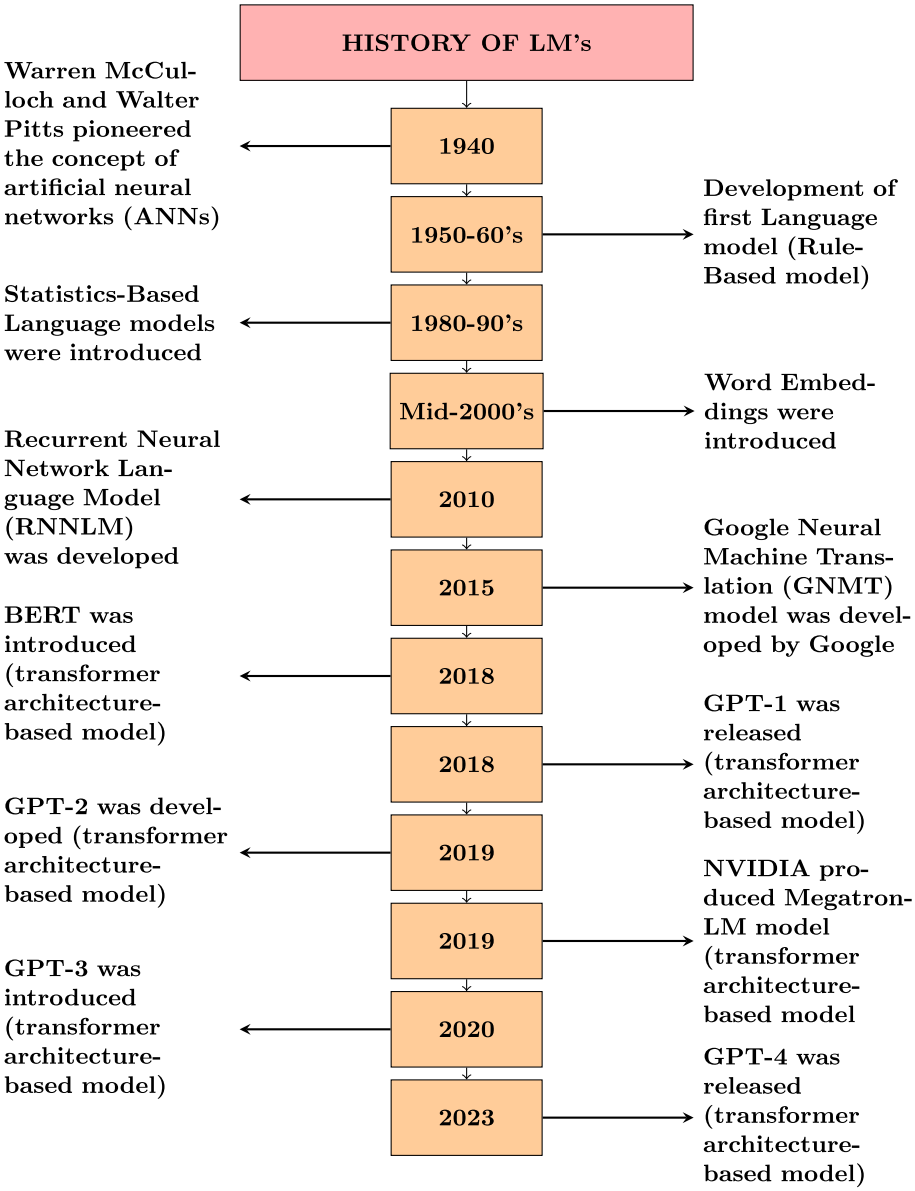

Fig. 3 Brief history of language models

2.1 Motivation for the survey

The rapid advancements in AI and LLMs, such as ChatGPT, have captivated the community, sparking a wealth of evaluations and discussions on their potential and impact. These technologies are reshaping various fields, from scientific text modeling [39] to text generation, healthcare, and education, as explored by [40]. This survey aims to delve deeper into the multifaceted applications of LLMs across different sectors. We are interested in understand-

ing their roles in finance and the labor market, as discussed by [41], and in supply chains, as explored by [42]. Furthermore, insights into their impact on telecommunications are sought and evaluated by [43], and their code-writing capabilities, as examined by [44]. The legal implications will also be considered, highlighted by [45], and their potential to address global challenges as outlined. The insights gained from this survey will contribute to a comprehensive understanding of the improvements in LLM pre-training, optimization, application, and competency assessment.

Although the amount of research on LLMs is increasing, studies on their technical nuances and efficient use still need to be done. A review paper with valuable applications will be highly beneficial to the subject, which is developing rapidly. Consequently, This work has been written as an application-focused evaluation as well. Our principal goal is exploring, acquiring, and assessing LMs in many areas. The underlying ideas of the LM are explored, several GPT family designs and other architectures are examined, and AI is generated. The limitations of the LLMs, such as the inability to recognize AI-generated text and the inclusion of context in Indian language learning, are also included in our thorough analysis. Furthermore, A collection of principles is offered to direct future investigation and advancement in the efficient application of LLMs. With more excellent knowledge and application of LLMs, this study can help.

The main signification contributions are as follows:

1. Presenting a detailed synopsis of LLMs, covering their technical aspects, developments, challenges, potential applications, and limitations, and identifying the detection of AI-generated text and the context of Indian languages in LLMs.
2. Evolution of LLMs (2018–2024): Providing a timeline of critical advancements in LLMs, focusing on significant models, architectural innovations, and their impact on NLP tasks.
3. Presenting a cutting-edge examination and contrast of several LLMs.
4. Taking up LLM-related ethical issues: Discuss their limitations, advantages, and disadvantages, including their limited understanding of the physical world, issues such as information hallucination, challenges with fine-tuning, and risks associated with foundational models.
5. Exploring applications of LLMs through practical use cases across diverse domains such as medical, financial, legal, political, media, entertainment, engineering, and chatbots.

The structure of this paper is organized as follows. Section 2 provides an overview of the historical development of LLMs and a list of the evolution of LLMs. Section 3 elucidates generative LLMs, including pre-training and fine-tuning approaches. Section 4 focuses on the training processes of LLMs and the risks of using LLMs. Section 5 discusses various types of pre-trained transformer-based LLMs and their architectures, while Sect. 6 elaborates on the specifics of transformer architectures. Section 7 explores the practical applications of LLMs, and Sect. 8 addresses the challenges related to detecting AI-generated text. Section 9 covers the context of LLMs in Indian languages. Section 10 highlights the advantages of LLMs, and Sect. 11 outlines their disadvantages. Finally, the paper concludes with recommendations for future research.

2.2 List of large language models

LLMs have become a cornerstone of AI, revolutionizing natural language understanding and generation. This study traces their evolution, beginning with foundational models like GPT-1 (2018) and BERT, through groundbreaking advancements such as GPT-3, to the latest innovations like Google's Gemini series and Meta's Llama 3.1. The paper delves

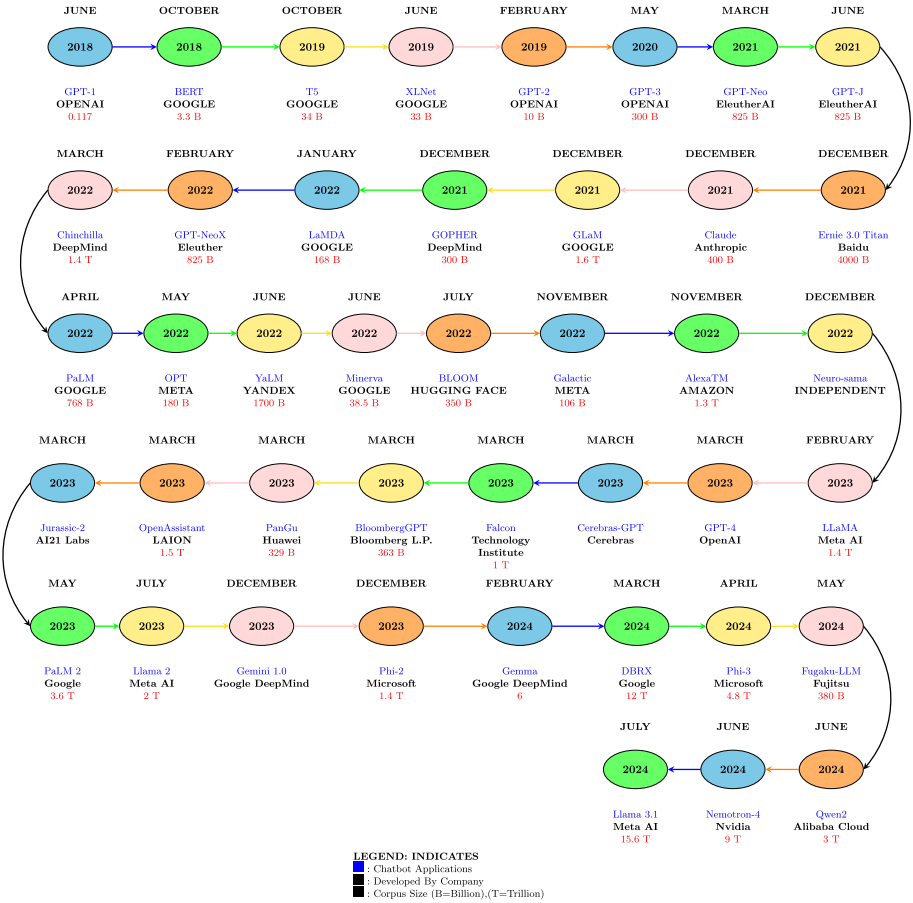

Fig. 4 List of large language models from 2018 to 2024

into the design principles, training methodologies, and performance benchmarks that have shaped these models. It offers insights into state-of-the-art (SOTA) technology and its future potential, as illustrated in Fig. 4. The GPT-1 model, published in June 2018 and developed by OpenAI, has 117 million parameters, has no corpus size, and is the encoder-only transformer, the first GPT model. The Bidirectional Encoder Representation from a Transformer (*BERT*) model, published in October 2018 by Google, is an early and influential LM with 340 million parameters and a corpus size of 3.3 billion words [23, 46]. The T5 architecture, introduced by Google AI in October 2019, represents a family of LLMs built on the Transformer framework. Adopting the encoder-decoder paradigm of the original Transformer, T5 utilizes an encoder to process input text and a decoder to generate the corresponding output text. The model includes up to 11 billion parameters and was trained on a corpus containing 34 billion tokens.

XLNet was introduced by [47] as an alternative to BERT, a model published in June 2019 developed by Google company, with a corpus size of ∼340 million parameters, is an encoder-only model that was trained on a dataset comprising 33 billion words. GPT-2 model published in the market in February 2019, developed by OpenAI, the corpus size employs a general-purpose model based on transformer architecture, with 1.5 billion parameters and 40

Giga Bytes (GB) (∼*10 billion tokens*) [37]. In May 2020, OpenAI released GPT-3, a powerful LM trained on 300 billion tokens and comprising 175 billion parameters [19]. Known for its coherent and natural-sounding text generation, GPT-3 has since become accessible exclusively via API, with no downloadable option available for local use as of 2024 [48].

In 2022, GPT-3.5, an enhanced version of GPT-3, was made available to the public through ChatGPT, an online chat platform [49]. The GPT-Neo model, published in the market in March 2021 and developed by EleutherAI, has 2.7 billion parameters and 825 GB. EleutherAI has provided the first in a series of free alternatives to GPT-3. While GPT-Neo was substantially poorer than the biggest GPT-3, it did better on several benchmarks than an equivalent-size GPT-3 device. GPT-J, or GPT-J-6B, is an open-source LLM created by EleutherAI in June 2021. This generative pre-trained Transformer model is designed to generate human-like text by extending a given prompt. It had a corpus size of 6 billion parameters and 825 GB [50], and it is an LM based on GPT-3. Ernie 3.0 Titan, a Chinese-language LLM, was released by Baidu, a Chinese multinational technology company, in December 2021. It boasts 260 billion parameters [51] and is trained on a 4 TB corpus. The Ernie Bot is built upon this model. The Claude model, developed by Anthropic, was published in December 2021. It has 52 billion parameters and a corpus of 400 billion tokens [52]. The model was fine-tuned for desirable conversation behavior.

Google published the Generalist Language Model (*GLaM*) in the market in December 2021. With a corpus of 1.2 trillion parameters, 1.6 trillion tokens, and a sparse combination of expert models, it was more costly to train than GPT-3 but less expensive to conduct inference. The Gopher model, developed by DeepMind, was published in December 2021. It has 280 billion parameters and was trained on a corpus of 300 billion tokens [53]; this model was further expanded into the Chinchilla model. Google launched the LaMDA (*LMs for Dialogue Applications*) model in January 2022. It was trained using a corpus of 1.56 trillion words and 168 billion tokens, including 137 billion parameters [54]. LaMDA is specifically designed to produce dialog-based replies. Developed by EleutherAI, the GPT-NeoX model was made available in February 2022. It was trained on a corpus of 825 GB [50] and contains 20 billion parameters [55]. The Megatron architecture serves as the model's foundation.

Google published the Pathways Language Model (*PaLM*) in April of 2022 [56] trained it using 768 billion tokens and included 540 billion parameters. Approximately 60 days were spent training the model with roughly 6000 TPU v4 chips. Developed by Meta, the Open Pre-trained Transformer (*OPT*) model was released into the market in May 2022. The corpus size was 175 billion parameters, 180 billion tokens [57], and GPT-3 architecture with a few Megatron-specific modifications. YaLM 100B, developed by Yandex LLC, a Russian technology company, was released in June 2022. This English-Russian LM is based on Microsoft's Megatron-LM architecture. It features 100 billion parameters and was trained on a 1.7 TB corpus.

Released in June 2022, the Minerva model was created by Google. The model, trained on a corpus of 38.5 billion tokens from online pages screened for mathematical content, includes 540 billion parameters. Minerva is intended to use step-by-step reasoning to solve scientific and mathematical problems [58]. Minerva has been refined using scientific and mathematical data based on the PaLM model.

In July 2022, Hugging Face spearheaded a partnership introducing BigScience's Large Open-science, Open-access Multilingual Language Model (*BLOOM*). It was trained using a corpus of 350 billion tokens (*1.6 TB*) with 175 billion parameters. Like GPT-3, BLOOM was trained using a multilingual corpus of diverse languages (*excluding programming languages*) and 30% English data. The Galactica model, developed by Meta and released in November 2022, featured 120 billion parameters and was trained on 106 billion tokens [59];

it was explicitly trained on scientific texts and modalities. The AlexaTM (*Teacher Model*), developed by Amazon and released in November 2022, featured 20 billion parameters and was trained on 1.3 trillion tokens [60]. It utilizes a bidirectional sequence-to-sequence architecture.

Neuro-sama, an AI VTuber and chatbot, was introduced to the market in December 2022. It is an LM specifically designed for live-streaming interactions on Twitch. Developed by Meta, the LLaMA model was trained on a corpus of 1.4 trillion tokens and published in February 2023 it has 65 billion parameters. It aimed to use a significant corpus of 20 languages for higher performance with fewer parameters. Stanford University researchers later trained an improved model called Alpaca, which was based on LLaMA weights.

GPT-4 is a multimodal LLM developed by OpenAI and represents the fourth iteration in the GPT series of foundation models. Released on March 14, 2023, it is rumored to have 1.76 trillion parameters [61]. GPT-4 is accessible to ChatGPT Plus users and has been integrated into various applications and products.

The Cerebras-GPT model, developed by Cerebras and released in March 2023, features 13 billion parameters and was trained using the Chinchilla formula. The Falcon model, introduced by the Technology Innovation Institute in March 2023, features 40 billion parameters and was trained on a corpus comprising 1 trillion tokens from Refined Web (*a filtered web text corpus*) as well as additional curated datasets [62]. Bloomberg L.P. developed and released the BloombergGPT model in March 2023; it has 50 billion parameters. A corpus of 363 billion tokens from Bloomberg's data sources and 345 billion tokens from general-purpose datasets were used to train it [63]. With a focus on financial data, our model performs strongly on general LLM benchmarks and beats previous models on financial tasks. The PanGu-Σ model was published in the market in March 2023, developed by Huawei company with 1.085 trillion parameters, and the corpus size was 329 billion tokens. The Open Assistant model, developed by LAION (*a German non-profit organization focused on open-source AI models and datasets*) and released in March 2023, features 17 billion parameters. It was trained on a corpus of 1.5 trillion tokens, utilizing crowdsourced open data.

Released in March 2023, the Jurassic-2 model was created by AI21 Labs, an Israeli startup specializing in NLP and creating AI systems for comprehending and generating natural language. This model speaks several languages. Google built the PaLM 2 (*Pathways Language Model 2*), launched in May 2023 with 340 billion parameters. It is utilized by the Bard chatbot and was trained on a corpus of 3.6 trillion tokens. Meta created the LLaMA 2 model, published in July 2023, with 70 billion parameters [64]. It replaced the original LLaMA model and was trained on a corpus of two trillion tokens.

Gemini 1.0 is a series of multimodal LLMs developed by Google DeepMind company. Released in December 2023, the model family includes three variants of different sizes. Gemini 1.0 is utilized in the chatbot bearing the same name and is designed to handle multimodal inputs effectively. Microsoft created the Phi-2 model, which was made available in December 2023. It has 2.7 billion parameters and was trained using a corpus of 1.4 trillion tokens. According to the publication, *"Textbooks Are All You Need"*, this so-called tiny LM was trained on *"textbook-quality*" data and is said to "match or outperform models up to 25 times larger." Using 96 A100 GPUs, the model training process took 14 days [65].

In February 2024, Google DeepMind published two versions of the Gemma model: one with two billion parameters and the other with seven billion. A corpus of six trillion tokens was used to train it.

DBRX is an open-source LLM developed collaboratively by the Mosaic ML team and Databricks. Released on March 27, 2024, it is a mixture-of-experts Transformer model with a total of 132 billion parameters, of which 36 billion are active per token. The model was

trained on a 12 trillion-token corpus at approximately 10 million USD cost, demonstrating its scale and investment in development.

Phi-3, developed by Microsoft and released in April 2024, is an LM comprising 14 billion parameters and trained on a corpus of 4.8 trillion tokens. Despite its substantial size and capabilities, Microsoft positions Phi-3 as a 'small LM', emphasizing its optimized and efficient architecture for specific use cases. IBM Granite, a series of decoder-only AI foundation models, was developed by IBM company and released to the market in May 2024. These models are designed to serve as a robust foundation for various AI applications. Fugaku-LLM, developed by Fujitsu in collaboration with the Tokyo Institute of Technology, was released in May 2024. This LLM comprises 13 billion parameters and was trained on a dataset containing 380 billion tokens. Notably, Fugaku-LLM is the largest model ever trained exclusively on CPUs, utilizing the Fugaku supercomputer, a testament to its pioneering approach to computational efficiency.

Qwen2, developed by Alibaba Cloud, a subsidiary of Alibaba Group, was released in June 2024. This LLM features 72 billion parameters and a corpus size of 3 trillion tokens. The Qwen2 series includes models of varying sizes, with the smallest containing 0.5 billion parameters, catering to diverse computational needs. Nemotron-4, developed by Nvidia, was released in June 2024. This model consists of 340 billion parameters and was trained on a corpus of 9 trillion tokens. The training process, conducted for a single epoch, utilized 6,144 H100 GPUs over a period spanning from December 2023 to May 2024. Llama 3.1, introduced by Meta AI in July 2024, features 405 billion parameters and was trained on a dataset comprising 15.6 trillion tokens. The 405B version required 31 million GPU hours on H100-80 GB GPUs, achieving a computational scale of 3.8×10^{25} FLOPs [66].

2.3 Statistical language model

SLMs belong to a class of methodologies that are applied in NLP and ML to find and measure statistical correlations and patterns in linguistic data [67], Applications, including speech recognition, text generation, spam detection, and predictive text input, all benefit from using SLMs. Because these models could quickly analyze enormous volumes of data, they were more accurate than early neural network approaches and rule-based systems. Despite their efficacy in many NLP applications, SLMs have difficulties comprehending linguistic context and predicting the semantic links between concepts. Long-range dependencies are frequently complicated for these models to represent [68, 69]. This model's principal goal is to anticipate a word's position in a sentence by analyzing the context of the word that comes before it. The algorithm predicts that after saying "*I love*," for instance, the following word may be "*you*."

These models operate on the principle of the Markov assumption, e.g., predicting the next word in a sequence based on recent context. SLMs with a fixed context length, such as "*N-gram*" models (*e.g., bigram and trigram models*), use this assumption to estimate probabilities within specific word sequences. To improve task accuracy, SLMs have found extensive applications in fields like IR [70] and NLP. Despite their effectiveness, they face a challenge known as the "*curse of dimensionality*," where accurately estimating higher-order models becomes complex due to the exponential increase in required transition probabilities. To address this data sparsity issue, smoothing techniques are commonly applied to improve model reliability, including back-off estimation [71] and Good-Turing [72] estimation.

2.4 Neural language model

Neural Language Models (*NLMs*) utilize neural networks, such as RNNs and Multi-Layer Perceptrons (*MLPs*), to determine word sequence probabilities [73–75]. Notably, the study in [75] introduced the concept of distributed word representations, establishing a word prediction function based on aggregated contextual features or distributed word vectors. Emphasizing extracting valuable characteristics from text input, A generic neural network technique capable of performing multiple NLP tasks simultaneously was developed. Additionally, the introduction of word2vec [76] simplified learning word representations using a shallow neural network. These approaches have proven highly effective across a range of NLP tasks. Beyond mere word sequence prediction, this research revolutionized the field of NLP by introducing LMs for representation learning.

2.5 Pre-trained language models

Pre-Trained Language Models (*PLMs*) undergo an initial pre-training phase, exposing them to many unlabeled texts. This allows the models to learn basic language structures like syntax, logic, vocabulary, and semantics. The wide range of NLP activities for which this complete LM may be used includes MT, text summarization, and question-answering systems. Fine-tuning models require them to be trained again on a smaller dataset tailored for a particular downstream task to maximize performance. This learning paradigm is known as "*pre-training and fine-tuning*." To further grasp the concept of "pre-training and fine-tuning," consider the following graphic example: According to martial arts literature, one must possess a strong foundation in internal martial arts, which may be attained via training in various methods to become a martial arts master. Then, using internal martial arts concepts, the person may rapidly and efficiently acquire a particular talent, like a sword or palm strike. This paradigm is the foundation for several PLM research efforts. It introduces various architectures, such as GPT-2, BERT [77–79].

2.6 Large language models

LLMs are trained using extensive text corpora, such as GPT-3, GPT-4, PaLM [19], and LLaMA [25]. These models are designed to enable systems to understand human instructions and adhere to human values. Rather than shifting to different domains, LLMs combine two fundamental processes: pre-training on a vast general-purpose corpus and fine-tuning to align with human values. Compared to PLMs, LLMs are very flexible and can shift from specialized to general-purpose models. More models, datasets, and computing power have led to notable improvements in various tasks and revealed unique features not present in smaller models. GPT-3, for instance, may use contextual information, but GPT-2 cannot. This indicates that GPT-3 uses task-related examples when presented with them to improve its problem-solving abilities. LLMs typically have over 100 billion parameters, with training data sizes ranging from a few hundred GB to several TB. For instance, the GPT-2 model, available in several versions, includes a variant with 1.5 billion parameters and requires 40 GB of text data for training. In comparison, the most giant version of GPT-3 has 175 billion parameters and needs 570 GB of text data. This comparison underscores the substantial differences in parameter scale and data requirements between LLMs and traditional LMs.

3 Generative LLMs

Generating text that closely mimics human writing is assigned to a category of NLP models called Generative language models. These models are engineered to comprehend textual input and furnish content suitable for the context. Their proficiency in discerning the statistical patterns and structures of language is facilitated by extensive training using extensive text datasets [19, 77]. For example, models like GPT-3 from OpenAI's GPT series are commonly employed within this domain. The conversion of input x to output y can be written as

$$P(y|x) = P(y|x; \theta) \tag{1}$$

Where θ is the parameter that controls the mapping between the input and the output.

In the second generation of NLP, probability plays a vital role in disambiguation, leading to a different formulation of mappings compared to earlier approaches. Statistical models from this period map inputs x to outputs y using learned weights, with human users providing input representations for x, often through feature vectors as described in [67]. In the third generation of NLP, the approach shifts to using tokens and positional encodings to represent inputs [18, 80]. This generation involves a two-phase process: The first phase focuses on establishing distinct learning objectives and utilizes self-supervised learning with unlabeled datasets [81]. This evolution highlights the transition from traditional statistical methods to more advanced NLP techniques.

One approach to achieving this involves different techniques for text prediction. First, a model predicts a masked segment of the text, similar to Masked Language Modeling (*MLM*) used in BERT [80]. Second, it predicts subsequent text, as utilized in decoder-only language models like GPT [46]. Third, it learns contextualized embeddings of words, akin to methods used in Word2vec, CBOW, and GLoVE [76, 82, 83]. The second phase involves fine-tuning the model using labeled datasets specific to particular tasks, which allows for further adaptation and optimization of the model's performance [81, 84].

While x consistently represents textual input (*which may be represented using embeddings in the third generation of NLP or features in the second generation*), y varies depending on the specific task. For instance, in sentiment analysis, y is a categorical variable indicating sentiment categories such as positivity, negativity, or neutrality [85]. In named entity recognition, y consists of categorical labels identifying whether a word at a particular position is part of a named entity and its type. In document question answering, y is a string of words extracted from a reference document. In MT, y represents a string of words in the target language [36]. Semantically, the variables x and y are not directly comparable across different tasks. For example, if x and y were sequences of words with x preceding y in a document, the nature of y would still depend on the specific task. The core principle of LLMs is that text generation, especially next-text prediction, is paramount [18].

A summary of LLMs and their relationships with other components is illustrated in Fig. 5. It provides a high-level overview of the key concepts related to LLMs. The training of an LLM begins with pre-training on massive unlabeled corpora, a crucial step for learning word prediction [46]. This phase results in a foundation model, which serves as the base for subsequent NLP tasks. Once pre-training is complete, the foundation model can be fine-tuned using a labeled corpus [84]. In this phase, the model is trained with input–output pairs, where the input is prompt, and the output is the desired answer. Fine-tuning allows the model to adapt and improve its performance on specific tasks by learning to generate appropriate responses for the inputs in the labeled corpus. This version emphasizes the sequence of training and fine-tuning processes and their roles in developing and applying LLMs.

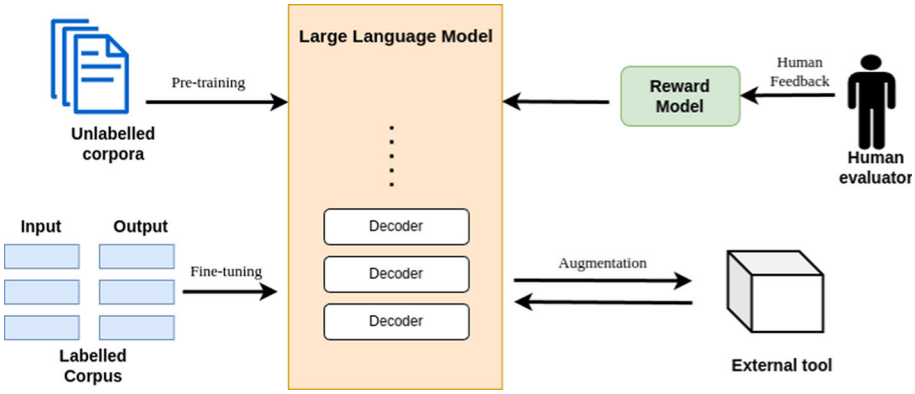

Fig. 5 Training an LLM with a stack of decoders

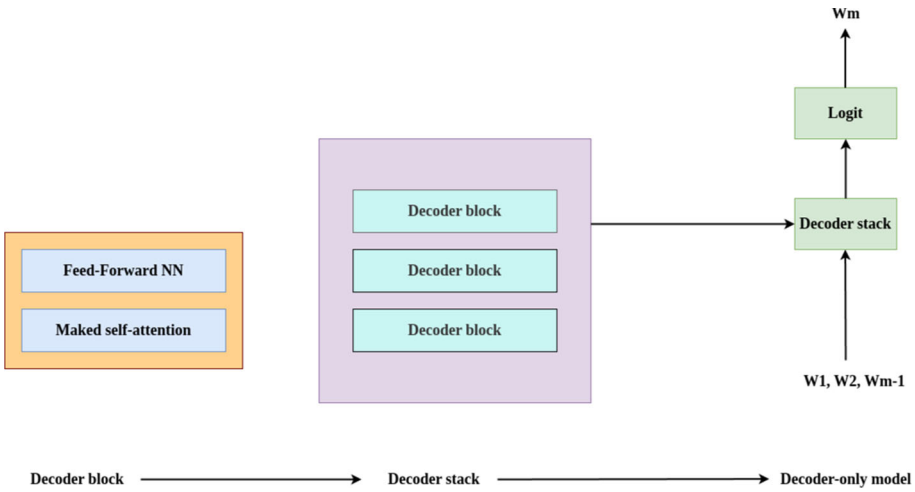

Fig. 6 Visualization of the evolution of a decoder-only model

3.1 Pre-training LLMs

The Transformer generates output text for MT by taking input text, such as a phrase in the original language. Encoders repeatedly employ self-attention to generate intermediate encodings of the input text. At the same time, decoders combine cross-attention and a modified version of self-attention to generate the output text [18]. Decoder-only models, which require only decoder blocks, utilize the transformer architecture's decoder as a fundamental component. This depiction illustrates in Fig. 6 the development of the decoder, which serves as a building block for LLM [46]. These decoder-only models utilize basic building blocks, such as self-attention and feedforward networks within the decoder block. Veiled self-attention is examined further, with Eqs. (2) and (3) detailing the masking of self-attention modulation [19].

$$\text{Attention}(Q, K, V) = \text{softmax}\left[\frac{QK^T}{\sqrt{d_k}}\right]V \tag{2}$$

where

- Q: Query matrix derived from the input token.
- K: Key matrix representing all tokens in the sequence.
- V: Value matrix containing the values associated with each token.
- d_k: Dimensionality of the query/key vectors.

The mechanism of masked attention is fundamental in autoregressive tasks such as text generation, where causality must be maintained to prevent future tokens from influencing current predictions. This mechanism is mathematically expressed as:

$$\text{Masked Attention}(Q, K, V) = \begin{cases} \text{Attention}(Q, K_i, V_i) & \text{for } 0 \leq i \leq m-1, \\ 0 & \text{for } i > m \end{cases} \tag{3}$$

Remember that matrix V is assigned a weight in Eq. (2) and as shown in Fig. 6. Consider the sentence, "*This bank offers all of its client's competitive interest rates*." For instance, K represents the critical matrix for each word in the phrase. Q is the query matrix for "interest." For each word in the phrase, QK^T represents a notion of similarity derived from the dot product of "interest." These similarity matrix components are normalized by d [18]. Then, softmax is used on all words in the sentence to provide a probability matrix representing the relative significance of "interest." Therefore, if the training data is good. The training regime is correct, relatively high values for the components indicate similarity between should be observed "interest" and "bank" and between "interest" and "customers" in the probability matrix [86]. In other words, the "representation for interest" is influenced by all surrounding words, particularly "interest" and "customers."

The causal masking in Eq. (3) ensures that predictions for a given position i depend only on the preceding tokens ($0 \leq i \leq m-1$). This design prevents information leakage from future tokens, which is critical in tasks like autoregressive text generation. However, while autoregressive masked attention is widely used, several alternative attention patterns have been proposed to improve efficiency, adaptability, and scalability across different tasks and sequence lengths. These include:

- *Full attention:* Full attention computes attention across all tokens in the input sequence, enabling bidirectional context aggregation. This pattern is commonly used in models like *BERT*, which require a global understanding of the input for classification and question-answering tasks [87]. However, its quadratic complexity, $\mathcal{O}(n^2)$, can be computationally prohibitive for long sequences.
- *Windowed attention:* Windowed attention restricts the computation to a fixed size window of neighboring tokens, significantly reducing the computational cost. For instance, models like *Longformer* utilize this approach to efficiently process long sequences, where local context suffices for many tasks. The complexity is reduced to $\mathcal{O}(n \cdot w)$, where w is the window size and $w \ll n$.
- *Dilated attention:* Expands the receptive field by attending to non-contiguous tokens at exponentially increasing intervals. This allows for a broader context while maintaining efficiency, making it suitable for structured or hierarchical data. The complexity is $\mathcal{O}(n \cdot \log n)$, balancing context size and computational cost [87].
- *Sparse attention:* Reduces computational cost by focusing on a subset of relevant tokens, either through pre-defined patterns (*e.g., BigBird*) or learnable mechanisms [88]. By focusing on relevant tokens, sparse attention achieves a complexity of $\mathcal{O}(|S|)$, where $|S| \ll n^2$. This makes it highly effective for long-sequence tasks, including document summarization and long-form text generation.

- *Global attention:* Combines global attention on specific key tokens with local attention for neighborhood tokens. This is useful in scenarios where some tokens (*e.g., [CLS] in BERT*) need a global view [89]. This hybrid approach balances the need for a global understanding with the efficiency of local interactions, making it suitable for tasks requiring both granular and holistic context.

Incorporating these alternative attention patterns allows models to balance better computational efficiency, context size, and task-specific requirements. A comparison of these patterns could provide valuable insights into their trade-offs and applicability across different domains. By acknowledging these diverse attention mechanisms, our study highlights the flexibility and adaptability of attention models, paving the way for future optimizations and innovations.

3.2 Fine-tuning LLMs

Generative AI is based on foundation models. LM must be modified to fit specific requirements, as in NLP [90]. Transfer learning is used to achieve this. The phrase "*transfer learning*" comprises two parts. The meaning of the word "*learning*" is comparable to "*main learning*." A labeled set is given to the model which it has to learn from. The phrase "transfer" suggests that instead of starting from zero, one should expand on the computational expertise provided in the foundation model and apply it to the specific task at hand [84]. Prompt-response pairs are used as the labeled set to fine-tune a model. There are three methods for fine-tuning: instruction response, sentence completion, and sentence labeling.
Sentence labeling: In which the answer is a category label attached to the sentence that serves as the prompt. By doing this, the prediction foundation model will be improved [91].
Sentence completion: Refers to the fine-tuning task where the models are trained to continue the prompted sentence. This will modify the foundation model to suit whole sentences based on an unlabeled dataset; the Gita GPT, for instance, illustrates the GPT model adjusted to match the Bhagavad Gita, one of Hinduism's[1] holy books. The resulting fine-tuned model employs the Bhagavad Gita for fine-tuning in addition to the GPT basis model.
Instruction responding: This approach focuses on fine-tuning models to respond to textual instructions with appropriate outputs. During fine-tuning on a labeled training set, the weights of the final layers of the model are updated, as illustrated in Fig. 7. The thick feed-forward layer receives signals from the penultimate layer, and the anticipated labels are returned in the layer's output. The model's optimization goal is to minimize the error between the expected and predicted labels (*as specified in the labeled set*). The weights of the bottom layers are frozen to preserve the foundational knowledge encoded in the model [92]. Foundation models can follow textual instructions by employing prompt-response pairs that consist of an instruction and an expected reaction. This methodology has proven effective in reasoning tasks and is referred to as instruction fine-tuning [93]. Advanced approaches, such as those utilizing rotary embeddings for improved positional encoding, have refined instruction-based adaptation for reasoning tasks [94].

3.3 Refining LLMs for conversational use

So far, LLMs primarily use text completion as their core learning task. They can be refined by fine-tuning using prompt-response pairs tailored to specific tasks. This process imparts two essential qualities:

[1] https://en.wikipedia.org/wiki/Bhagavad_Gita.

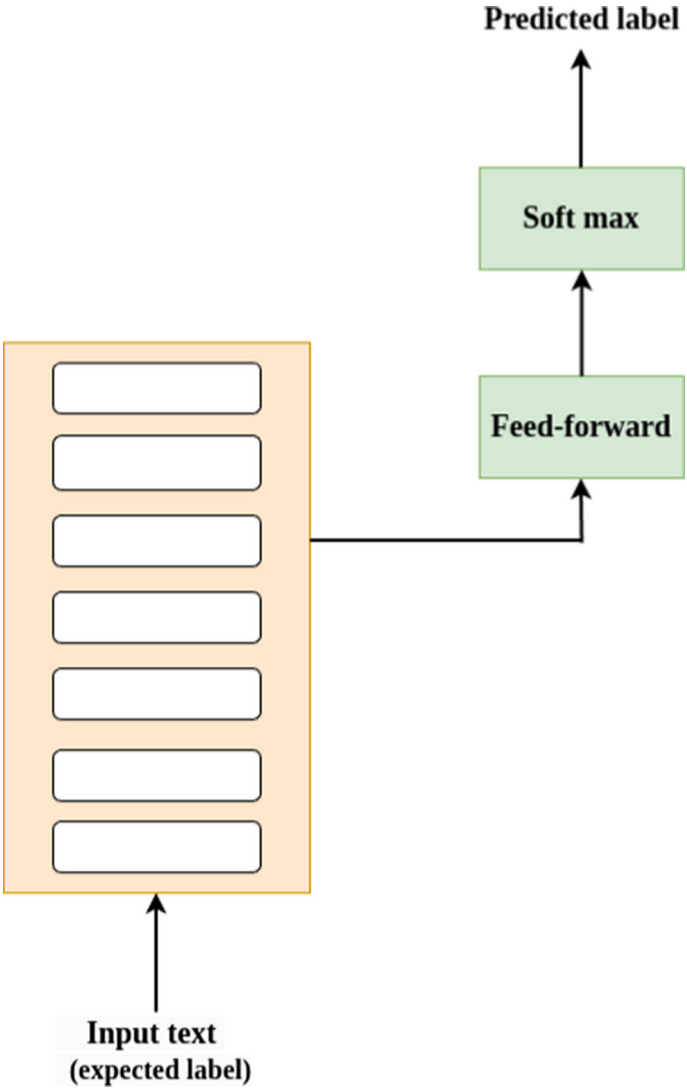

Fig. 7 Fine-tuning GPT for classification tasks

- Prediction of Next Words: LLMs can predict the next word in a sequence based on the foundation model's prior training [46].
- Word Differentiation: LLMs can discern between various potential words through fine-tuning, adjusting their predictions based on specific tasks or domains.

However, are these characteristics sufficient for an LLM to effectively converse with humans? Since human conversations are discourse-based, additional capabilities are required. Specifically, an LLM must understand and interpret any agreements or contextual nuances expressed by the human communicator. A human's relevant query or context can clarify the discussion and improve interaction, making conversational capabilities crucial for engaging human-like dialog. This is where reinforcement learning becomes beneficial. Reinforcement learning

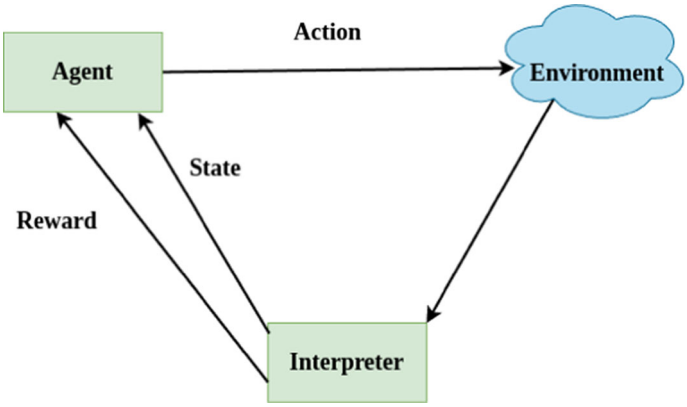

Fig. 8 Components of reinforcement learning

enables an AI agent to acquire new skills through iterative learning from its interactions. A typical reinforcement learning agent is composed of five key components, as illustrated in Fig. 8:

1. Environment: The setting in which the agent operates [95].
2. State: The agent's perception or understanding of its environment [96].
3. Action: The decisions made by the agent to affect its environment
4. Reward: The feedback the agent receives reflects the impact of its actions on the environment.
5. Policy: The agent's strategy to decide its actions based on the received rewards.

Enhancing of LLMs with external tools

The dependence of LLMs on sentence completion is often a source of factually inaccurate responses. Answering questions involves more than simply finishing sentences. Mathematical computation is one example. If the question is, "I have four bananas and three apples in two boxes," the model doesn't need to search for instances that include apples and bananas to know how many fruits do I have?. When people answer the question, they consider two factors: (1) Three and four are numbers, and (2) apples and bananas are fruits, but boxes are not. Thus, to respond to the questions, the number of fruits must be "*added*." This comprehension goes beyond language alone. It includes comprehending the relationship between the mathematical action of addition and "*how many*." Thus, LMs must consider information sources other than "*language*" or "*text*" in the strict sense.

Consider a different scenario. Suppose a user asks a natural language question to the calendar app on their mobile device: "When am I meeting my client at Ajax Technologies"? (*Ajax Technologies is a fictional name; any resemblance to actual entities is purely coincidental*). A purely language-based model might respond with the incomplete phrase, "*You have a meeting with Ajax Technologies on....*" However, the specific details required to complete the sentence, such as the meeting time, must be retrieved from the application's database, i.e., the user's calendar [19].

As LMs become incorporated into more complex applications, like calendar systems, they can no longer depend solely on text completion tasks based on language modeling.

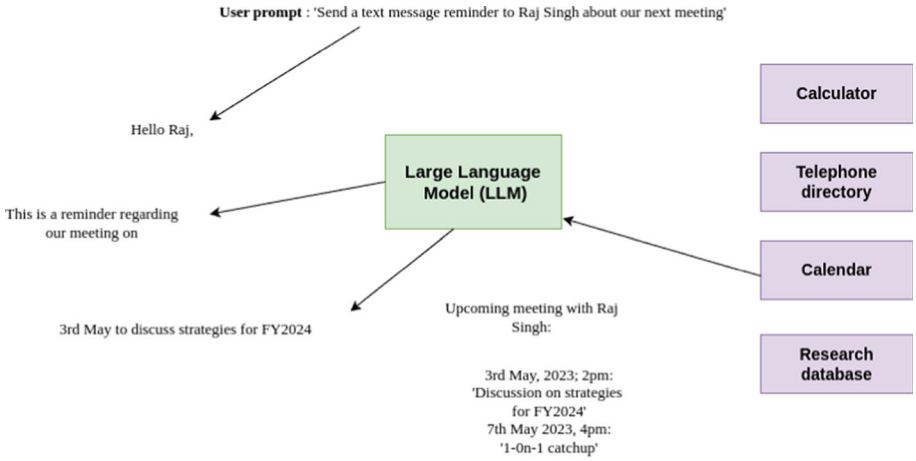

Fig. 9 Illustrative example of augmented language models

Instead, they need to interact with additional information sources or external tools. Augmented language models represent a step in this direction [97]. These models integrate generative language models with external tools and databases. Examples include a calculator program (as discussed in the above fruit-counting scenario) or a search engine for retrieving information when the model lacks confidence. In cases like the calendar application, the necessary database can be directly connected to the system, or a knowledge base (*such as a structured repository containing geographical data*) may be utilized.

As a result, the learning objective evolves to the following:

Given the current input, predict the next word (OR) and generate a call to an external tool to retrieve relevant information. The data retrieved from the tool serves as additional context for generating the subsequent word(s).

Figure 9 illustrates an example of an augmented language model in use. Take the prompt: *Send a text reminder to Raj Singh about our next meeting*. The LLM can access various tools, such as a calculator, contact list, calendar, or research database. Using its text generation capabilities, the model generates the message: *Hello Raj, this is a reminder about our meeting on*.... The words *Raj*' and *reminder*' are generated directly from the input prompt. Afterward, the LM identifies the need for information about the upcoming meeting and consults the calendar. It retrieves details about meetings with Raj Singh, and this information (*such as the date and title*) is used as context to complete the message, for example,... 3rd May to discuss strategies for FY 2024.

4 Training large language models

For LLMs to thrive, several crucial processes must be developed effectively [98]. As seen in Table 1, this usually entails gathering and pre-processing enormous volumes of text data from many origins, including literary works, journals, websites, and various other text sources [99]. Carefully chosen datasets are the basis for LLM training [100]. Assuming that language generation is a random process, unsupervised learning is used in the training phase to train the model to anticipate the next word in a sequence based on the previous context. Before training,

Table 1 State-of-the-art training pipeline for LLMs

Phase	Pre-training	Supervised-finetuning	Reward modeling
Data collection	Unprocessed internet data II	Illustration	Evaluations
Computational method	Text prediction modeling	Text prediction modeling	Two-class classification
System	Foundation model	Supervised fine-tuning model	Reward modeling
Assets	Hundreds of GPUs	1 to 100 GPUs	1 to 100 GPUs
	and months of training	and days of training	and days of training
	Suitable for deployment	Suitable for deployment	Not suitable for deployment

harmful, noisy, and duplicate data are removed, and precautions are made to minimize privacy [92].

Transformers are now necessary for LLMs to work and allow them to simulate long-range dependencies, interpret textual data [101], and produce new content that conforms to the author's or genre's style and traits [53]. By fine-tuning the model's parameters, the goal of training is to increase the likelihood that, in a particular environment, the model will provide the right word. For this kind of optimization, backpropagation is usually combined with algorithms such as versions of Stochastic Gradient Descent (*SGD*) [102], which compute gradients iteratively to update the model's parameters. Details about several well-known transformer-based LLMs are provided in the Sect. 5.

4.1 Risks of using LLMs

Utilizing LLMs introduces three primary risks: hallucination, bias, and deception. Due to their training in next-word prediction, LLMs may occasionally produce incorrect information, a phenomenon known as hallucination [103]. It is crucial to advise users of LLM-based technologies to verify the accuracy of generated outputs. Augmented LLMs are designed to minimize hallucinations by making API calls to access factual data. However, this approach can also lead to API hallucination, where the model initiates unnecessary API requests even with access to accurate information. The training data for foundational models often includes biased or harmful content, which can result in LLMs generating outputs that reflect these biases, including racist, sexist, or homophobic elements. This issue must be carefully considered when deploying LLMs for commercial use, with debiasing techniques and harmful content detection as potential strategies to mitigate these effects. Lastly, because LLMs generate coherent, fluent text, they present a risk of being used to produce deceptive content for activities like phishing and scams. Such applications highlight potential misuse, underscoring the need for Internet users to remain vigilant against deceptive LLM-generated content.

4.1.1 Mitigating risks of malicious use of LLMs

- *Model access control:* Limiting access to LLMs can help reduce the risk of misuse. One potential solution is implementing controlled API access, where organizations can monitor, restrict, and block suspicious [104] usage patterns.
- *User verification and access policies:* Platforms providing LLM access can implement strict user authentication, requiring verification processes to ensure the user's identity and intent is legitimate. Additionally, implementing usage policies restricting certain behavior types can discourage malicious users.
- *Detection and monitoring systems:* Developing tools to detect and flag suspicious or harmful outputs [105] in real-time can be a proactive measure. These tools would analyze the content generated by LLMs, using automated systems to identify and mitigate risky or abusive use cases (*e.g., identifying phishing or disinformation content*).
- *Adversarial testing and robust model design:* Continuously testing LLMs with adversarial prompts [106] can help reveal vulnerabilities and allow for more robust model adjustments. Techniques such as reinforced training to discourage harmful outputs and integrating adversarial defense mechanisms within LLM training can help make these models less susceptible to misuse.
- *Awareness and education:* Encouraging awareness around the ethical use of LLMs and educating users on the risks and consequences of misuse [107] can be another layer of defense. Partnering with institutions to establish guidelines and ethical standards can reinforce responsible usage.
- *Prompt engineering and guardrails:* Embedding guardrails within the LLM's response generation pipeline, such as filtering prompts that resemble phishing attempts or queries intended for harmful purposes [108], can help prevent inappropriate usage [109]. Advances in prompt engineering could also restrict models from responding to potentially harmful prompts.

5 Transformer-based LLMs

5.1 Bidirectional encoder representations from transformer

BERT, introduced by [80], is a significant LM known for its proficiency in advanced NLP tasks. Compared to earlier leading models, BERT's encoder-only transformer architecture represents a notable advancement. A 2020 literature review claims that BERT has evolved into a common baseline for NLP experiments, with more than 150 studies devoted to its analysis and enhancement in less than a year. Initially, two model sizes, BERT BASE and BERT LARGE, were used to analyze English language data. Twelve encoder layers, each with 12 bidirectional self-attention heads, make up BERT BASE, which has 110 million parameters. BERT LARGE features twenty-four encoder layers, 16 bidirectional self-attention heads per layer, and 340 million parameters. As reported by [110], both models were pre-trained on the 800 million-word Toronto Book Corpus and the 2.5 billion-word English Wikipedia. Pre-training and fine-tuning are the two primary phases of BERT's training procedure [111]. BERT gains the ability to represent language from a sizable corpus of unlabeled text data during pre-training. MLM and Next-Sentence Prediction (*NSP*) are the two main tasks used to do this [112, 113]. MLM masks some of the input tokens to assist the model in grasping context from both sides. The model is trained to predict these masked tokens. NSP entails

training BERT to recognize if a sentence comes after another to improve its comprehension of sentence coherence [114].

Using labeled data, BERT is fine-tuned on specific tasks following pre-training. BERT's acquired representations are fine-tuned to specific tasks such as named entity identification and sentiment analysis. This step updates the model's parameters using gradient descent optimization and backpropagation. This enables BERT to function successfully on the intended tasks by modifying its weights following the task-specific labeled data. It takes a lot of computer power to train BERT, and robust hardware like GPUs, Tensor Processing Units (*TPUs*) [115], or Field-Programmable Gate Arrays (*FPGAs*) [116] are frequently used. Furthermore, BERT is much more adept at capturing intricate dependencies and distant contextual linkages because of built-in transformer architectural features, including layer normalization, residual connections, and attention mechanisms [117].

5.2 eXtreme language understanding network

The constraints of conventional unidirectional language modeling approaches were addressed by [47] with the introduction of XLNet, an enhanced pre-training approach. Using a permutation-based training technique, XLNet can capture bidirectional context better than typical auto-regressive models [118]. XLNet does not rely on a fixed sequence of preceding words to predict the next word. Instead, it considers all possible permutations of the input sequence, modeling the probability of each permutation. This approach allows XLNet to capture dependencies in both forward and backward directions, thereby mitigating the limitations associated with traditional sequential modeling. There are two main training phases for XLNet: supervised fine-tuning and unsupervised pre-training. In unsupervised pre-training, XLNet maximizes the expected log-likelihood over all possible permutations of the input sequence to learn to predict words based on the whole context. XLNet can capture bidirectional context more successfully owing to this permutation-based method [119]. The pre-trained model is then supervised and fine-tuned using labeled data to fit specific tasks.

This is accomplished by utilizing a transformer architectural variant similar to models such as BERT. One characteristic of XLNet's training methodology is the permutation-based goal function. Specific difficulties arise because XLNet training uses a permutation-based objective function. XLNet must consider every potential combination, unlike conventional autoregressive models that make predictions based on the causal order of words [120]. As the number of training examples increases exponentially, training becomes computationally expensive. Effective methods like "*factorized sampling*," which samples a portion of permutations for each training cycle, are needed to address this issue. The significant processing resource needs of XLNet make training a difficult task. Due to the enormous number of potential permutations and the large size of the model, there is an increase in memory and processing demands. As a result, distributing training across several GPUs or TPUs is frequently necessary for XLNet, which can lengthen the training period [121].

5.3 Speech-XLNET

Speech-XLNet, developed by [47], utilizes a Self-Attention Network (*SAN*) and is fine-tuned within a hybrid SAN/HMM architecture to train unsupervised acoustic models for learning speech representations. This technique leverages its attention mechanisms to emphasize global structures, serving as a powerful regularizer that enhances the performance of the SAN. Speech-XLNet further improves the efficacy of speech representation learning by allowing

the model to investigate bidirectional settings. Comparing Speech-XLNet to systems trained using randomly initialized weights, experimental findings on the TIMIT and WSJ datasets show a considerable improvement in the convergence speed and recognition accuracy of the SAN/HMM system. Using the TIMIT and WSJ tasks, the model shows the most significant relative improvement, achieving 11.9% and 8.3%, respectively. The top-performing system on the TIMIT test set has an impressive 13.3% Phone Error Rate (*PER*). As far as is known, this is the lowest PER that a single system has ever achieved.

5.4 DIALOGXL

DialogXL, developed by [122], offers improvements to address more complex historical contexts and multiparty conversation structures. The first step in improving XLNet's conversational data modeling effectiveness is to switch from segment-level to utterance-level handling of recurrence. Second, the incorporation of dialog-aware self-attention allows for capturing critical dependencies between and among speakers, in contrast to XLNet's normal self-attention. Four ERC benchmarks are used in a complete set of studies that compare the DialogXL with standard models during training. All datasets show that DialogXL performs better than the baseline models, as consistently demonstrated by the experimental findings.

5.5 Text-to-Text Transfer model

The Text-to-Text Transfer Transformer (*T5*) model was created by Google and is trained in a "*text-to-text*" environment. T5's main innovation is framing every work as a text generation challenge. To unify the model's handling of different issues entails translating multiple tasks such as question answering, summarization, and text classification into a text-to-text format [123, 124]. For instance, rather than training T5 to provide direct answers to queries, it is trained to generate a comprehensive response by incorporating the given query and relevant context. During its pre-training phase, T5 utilizes a modified version of the transformer architecture. The pre-training objectives generally involve predicting the target text from the source text, with maximum likelihood estimation as the foundation. After pre-training, T5 is fine-tuned on specific downstream tasks to adapt the model to various applications [91].

5.6 BIOGPT

BioGPT is a large-scale language model designed explicitly for biological data, developed by the Allen Institute for AI (*AI2*) [125]. Built on the GPT architecture, BioGPT was trained on an extensive corpus of biological literature, including full-text articles and PubMed abstracts. Studies have shown that BioGPT performs better than other biological LMs on tasks including relation extraction, named entity identification and query responding. Because the pre-trained model weights are available to the public, researchers may maximize BioGPT using their biomedical text data [126]. By enabling more precise and compelling analysis of substantial amounts of biomedical text data, BioGPT can potentially progress biomedical research considerably [127]. Pre-trained LLMs are fundamental to NLP and offer a base for various applications without requiring much initial training. Their extensive usage, sophisticated language creation, and comprehension skills make them very effective. To get impartial and equitable results, it is crucial to utilize these models responsibly and consider ethical issues.

5.7 Conditional Transformer Language Model

Based on predefined prompts or control codes, [128] created a language model called the Conditional Transformer Language Model (*CTRL*) to generate text. CTRL is unique because it conditions the text generation process through control codes. Users can set the generated content's topic, style, and desired features using these control codes. They guide the model's behavior during inference and training in unambiguous directions. According to [129], the fine-tuning phase of CTRL plays a crucial role in tailoring the model to particular domains or tasks, enabling it to generate text that aligns with specific requirements or preferences.

5.8 Robust optimization of BERT pre-training approaches

RoBERTa, a variant of the BERT model, was developed by Facebook AI researchers. Like BERT, RoBERTa employs self-attention, a transformer-based mechanism, to evaluate input sequences and generate contextualized word representations. However, RoBERTa distinguishes itself significantly from BERT regarding its training process and dataset size. RoBERTa was trained using a more efficient training process and a substantially larger dataset. Specifically, it leveraged a dataset containing 160 GB of text, over ten times larger than the one used to train BERT [130].

Additionally, RoBERTa incorporates a dynamic masking strategy during training, enhancing the model's ability to learn more robust and universally applicable word representations. Numerous studies have showcased RoBERTa's superior performance to BERT and other SOTA models. RoBERTa has demonstrated exceptional performance over BERT and other cutting-edge models across various NLP tasks such as text categorization, question answering, and language translation [89]. RoBERTa is a popular choice for research and business applications and has also served as a foundational model for several other successful NLP models [131]. RoBERTa is a robust and efficient LM that has significantly advanced various applications and pushed the boundaries of NLP.

5.9 Open-source versus closed-source LLMs

Users must make a crucial choice when implementing LLMs for applications between closed-source and open-source LLMs. Each category presents unique hazards and obstacles to consumers, with advantages and disadvantages. Open-source LLMs greatly enhance the community's ability to innovate and adopt new technologies. Although they provide less assistance, they give users greater control and transparency over the models. With open-source LLMs, customers have greater flexibility to modify and tailor them to their requirements.

Nonetheless, users of open-source LLMs typically rely on the community for assistance. A lack of robust internal technical resources may make it difficult for them to adapt, customize, and integrate LLMs, increasing the possibility of a failed rollout. On the other hand, closed-source LLMs can provide consumers with more committed assistance, reduced transparency, and model control. Large companies with significant resources to assist development, enhancement, and implementation frequently support closed-source LLMs. However, because closed-source LLMs need more power and transparency, users may need help to tailor them to their needs. Additionally, users may risk vendor reliance and sensitive information being leaked when submitted to the LLMs.

Moreover, there may be significant differences in the expenses associated with using open-source vs closed-source LLMs. While utilizing closed-source LLMs may require less

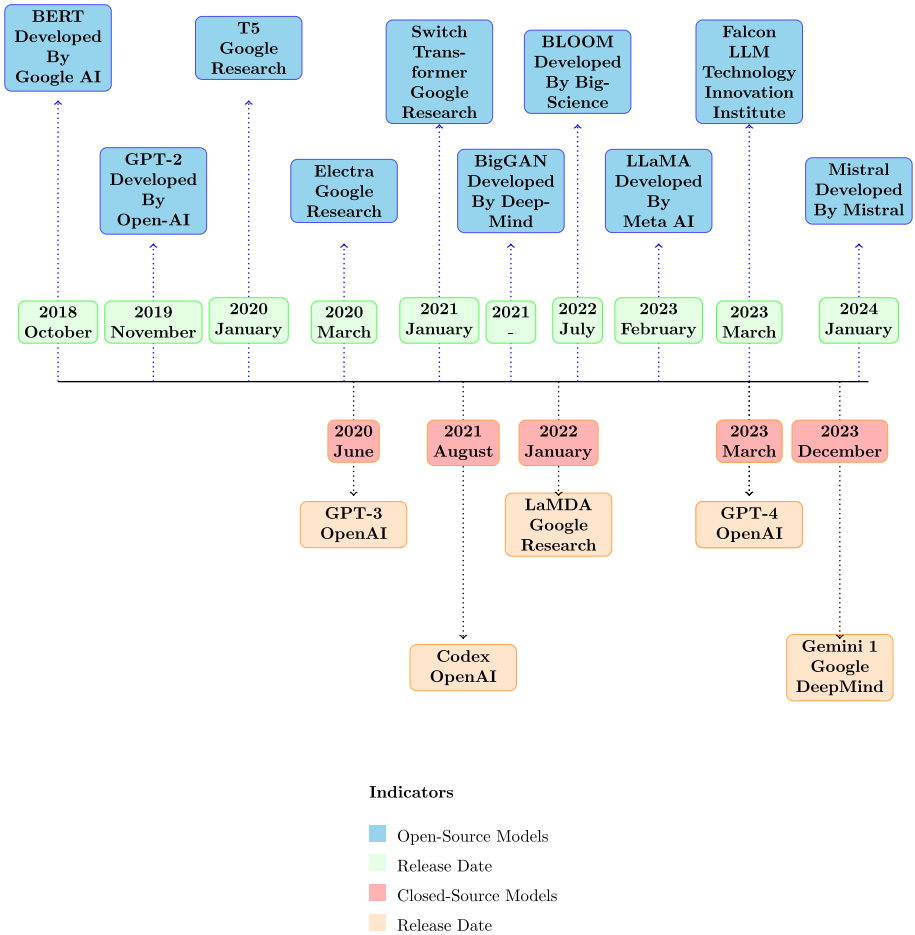

Fig. 10 Open-source and closed-source language models

initial investment but higher subscription expenses that arise due to vendor price fluctuations, implementing open-source LLMs may demand a more significant initial investment but fewer predictable long-term expenditures. The users' particular demands, goals, and resources determine whether to utilize closed-source or open-source LLMs. Before choosing, consumers must thoroughly weigh each solution's advantages and disadvantages [132].

Life Cycle Management (LCM) is changing quickly as more organizations create closed- and open-source models, as shown in Fig. 10. A breakdown of significant LLMs by year, divided into open-source and closed-source models,

- *Open-source models by year*: BERT, developed by Google AI, was a notable open-source model released in October 2018 [78]. In February 2019, OpenAI introduced GPT-2, initially releasing it with limitations due to concerns about misuse. The model was made available as open-source software by November of the same year [77].
  In 2020, several significant models were introduced. Google Research released T5 in January [91] and Electra in March as open-source models [133]. Another notable open-source architecture, the Switch Transformer, emphasizing efficiency and scalability, was

introduced by Google Research in January 2021. The same year saw the release of BigGAN by DeepMind, which advanced the capabilities of generative models [134]. In 2022, BigScience launched the open-source model BLOOM, which became available in July [135]. Several significant releases occurred in 2023. Meta AI introduced the LLaMA model in February [25], and the Technology Innovation Institute (*TII*) unveiled the Falcon LLM in March. Continuing the trend of accessible and community-driven development, Mistral released the Mistral model as open-source software in January 2024.

- *Closed-source models by year*: In June 2020, OpenAI released the GPT-3 model as a closed-source system through a for-profit API [19]. This was followed by the release of Codex, another closed-source model by OpenAI, in August 2021, designed explicitly for code generation [44]. Google Research introduced LaMDA as a closed-source model in January 2022 [136]. The closed-source landscape saw notable additions in 2023, with the release of Anthropic's Claude in March and OpenAI's GPT-4, also introduced in March. Additionally, Google DeepMind made Gemini 1 a closed-source model through a commercial API in December 2023.

6 Transformer architecture

LLMs are fundamentally built on the Transformer architecture [18], which efficiently processes sequential data without iterative approaches. Utilizing an attention-based system to recognize input–output connections across the entire sequence. The model is flexible enough to accommodate diverse sequence lengths since it can handle inputs of different lengths and change its emphasis accordingly. Because of its versatility, the transformer design has emerged as the industry standard in several domains, frequently replacing more intricate convolutional networks or RNNs with more effective configurations [137]. This is especially crucial for applications including LLM.

Transformer architecture typically employs three main architectures: encoder-only, decoder-only, or encoder-decoder. The transformer architecture, covered in depth in this section, is the foundation for most of these models. Figure 11 shows the transformer model's design. The seven primary parts of the transformer model are explained individually below.

6.1 Input embeddings

Tokens are textual elements, such as words, subwords, or characters, that constitute the training data for ML models [138]. While these models are designed to process numerical data, textual inputs must first be converted into a numerical format. To achieve this, **tokenization** is performed. Tokenization divides the incoming text into smaller, relevant parts called *tokens*. Each token is then assigned a unique numerical identity, bridging linguistic data and numerical representations. These numerical representations are referred to as **input embeddings**, numerical representations of tokens in continuous vector space. These embeddings allow the ML model to understand word meanings by positioning them within a mathematical space where semantically similar tokens are located close to each other. For instance, embeddings enable the model to associate words with similar meanings by generating vectors of comparable structure. This property results from the training process, where the model learns to encode relationships between tokens. The structure and role of input embeddings in the Transformer architecture are depicted in Fig. 11.

In the Transformer architecture, input embeddings are composed of two main components:

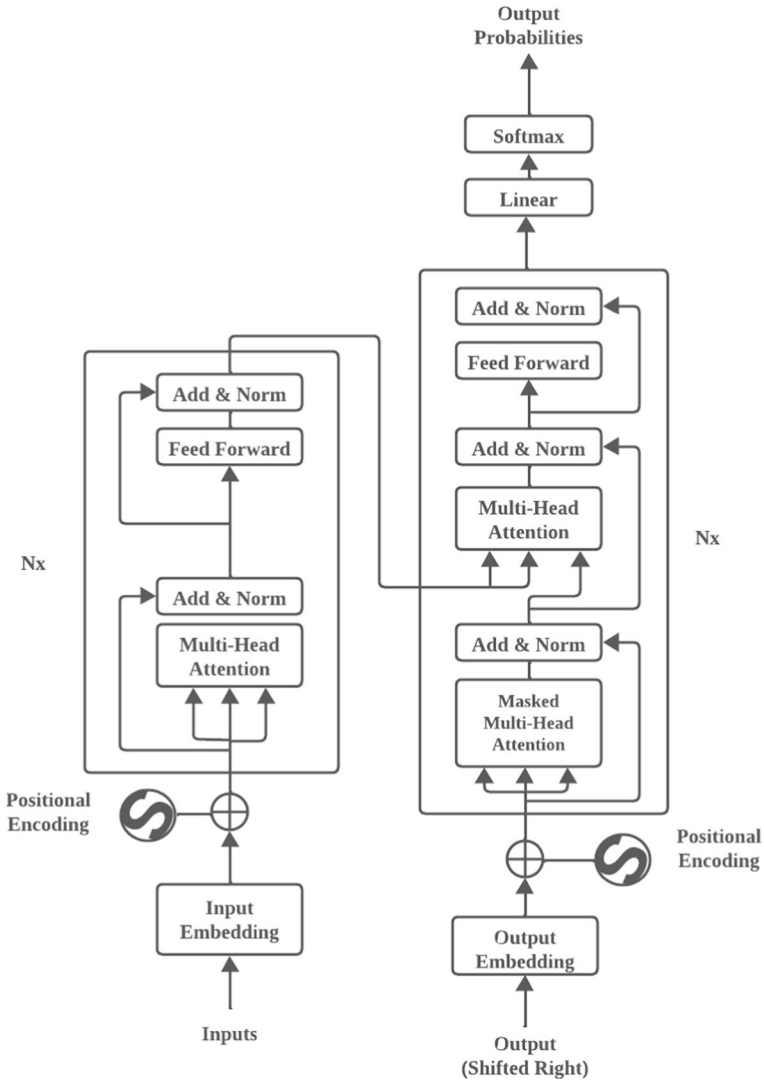

Fig. 11 Transformer architecture courtesy of [18]

- *Token embeddings*: Representations of individual tokens in a dense, continuous vector space.
- *Positional encodings*: Additional vectors encode positional information to maintain order in sequences.

This combination ensures that semantic and positional information are preserved when processing text. Let the input sequence be represented as $\{x_1, x_2, \ldots, x_n\}$, where x_i is the i-th token in the sequence. Each token x_i is mapped to a token embedding vector $\mathbf{E}_i \in \mathbb{R}^d$ through a look-up in a trainable embedding matrix $\mathbf{E} \in \mathbb{R}^{V \times d}$. Here:

- V is the vocabulary size.
- d is the embedding dimension.

Fig. 12 Define $d_{\text{model}} = 512$, which represents the size of the embedding vector of each word

The embedding vector $\mathbf{E}_i$ for token x_i is computed as:

$$\mathbf{E}_i = \mathbf{E} \cdot \mathbf{e}(x_i) \tag{4}$$

where

- $\mathbf{e}(x_i)$ is a one-hot encoded vector of size V, representing token x_i.
- $\mathbf{E}_i$ is the resulting token embedding, a continuous vector of size d.

Semantic representation of Input embeddings maps tokens into a high-dimensional vector space where semantically similar tokens are closer. This property allows the model to effectively capture the meaning and context of tokens. For example, as illustrated in Fig. 12, the phrase *YOUR CAT IS A LOVELY CAT* is tokenized into six tokens: *YOUR*, *CAT*, *IS*, *A*, *LOVELY*, and *CAT* (*repeated*). These tokens are transformed through the following steps:

1. *Input IDs assignment*: Each token is assigned a unique identifier based on its position in the vocabulary, e.g., *YOUR* (105), *CAT* (6587), *IS* (5475), *A* (3578), *LOVELY* (65), and *CAT* (6587) again for the second occurrence.
2. *Embedding vectors generation*: Each Input ID is mapped to a dense embedding vector of fixed size $d = 512$. For instance, the embedding vector for *YOUR* (ID: 105) might start with values like 952.207 and 5450.840 and continue until it forms a 512-dimensional vector.

These embeddings enable the model to process and understand linguistic data in numerical form, which is suitable for downstream tasks. Transformer models can use input embeddings to compare semantic similarities between tokens, phrases, or documents. By converting text into vector representations, the model can measure similarity using metrics such as cosine similarity, enabling applications like search and classification.

6.2 Positional encoding

Understanding the sequential order of words in a sentence is crucial in NLP, as word order often carries significant semantic information. Changing the arrangement of words can alter their meaning entirely. While traditional architectures like RNNs, LSTMs, and MLPs inherently account for the order of inputs due to their sequential nature, the transformer model does not natively encode positional information because it relies on the self-attention mechanism. To address this limitation, the "*Attention is All You Need*" introduced **positional encodings**, which numerically represent the position of each word in the sequence. This encoding allows the model to incorporate sequential context effectively. The original sinusoidal positional

encodings are computed as follows:

$$\mathrm{PE}(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \tag{5}$$

$$\mathrm{PE}(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \tag{6}$$

Here, pos is the position index, i is the dimension index, and d_{model} represents the dimensionality of the embeddings. These sinusoidal functions ensure that the positional encodings can generalize to sequences longer than during training, as the positional pattern depends on fixed mathematical functions rather than learned parameters. By enriching input embeddings with positional encodings [139], the transformer model captures the order of tokens and generates grammatically and semantically accurate outputs. However, the NLP community has progressed beyond these fixed sinusoidal encodings, introducing several alternative methods to address their limitations. One such advancement is **Learnable Positional Embeddings**, which treats positional information as trainable parameters learned during the training process. Unlike fixed encodings, learnable embeddings allow models to adapt to specific datasets and tasks. Models like BERT have demonstrated the effectiveness of this approach.

Another prominent method is **Relative Positional Embeddings** [140], which focuses on the relative distances between tokens instead of their absolute positions. This technique is particularly effective in scenarios where relative order carries more importance, such as language modeling tasks. Models like Transformer-XL and T5 adopt this approach to improve long-range dependencies in sequences.

For scaling to longer sequences, **Rotary Positional Embeddings** (***RoPE***) [94] have been introduced, which map positional information into a complex space and use rotation-based transformations to encode relative positions. RoPE enhances efficiency and scalability, making it suitable for modern large language models like GPT-4. Similarly, **Dynamic Positional Bias** allows models to learn task-specific positional patterns without explicitly encoding positions, integrating positional information implicitly within the attention mechanism.

In addition, some long-sequence models, such as *Linformer*, have explicit positional embeddings. Instead, they incorporate positional information directly into the attention mechanism, simplifying computations while maintaining the ability to capture positional dependencies. These advancements in positional encoding techniques reflect transformer-based architectures' growing adaptability and efficiency.

6.3 Encoder

An encoder plays a crucial role in processing the input text within the neural network by generating a series of hidden states that capture the meaning and relationships within the input sequence [141]. In the transformer architecture, the encoder is built by stacking multiple layers, each consisting of a self-attention mechanism followed by a feed-forward neural network. Each layer includes additional architectural features: layer normalization and skip (*residual*) connections. The self-attention mechanism captures dependencies between words in the input sequence by computing attention scores for each word relative to every other word, allowing the model to weigh the importance of each word in context. This enables the encoder to understand relationships across the entire sequence, regardless of distance. Following the self-attention layer, each encoder layer contains a feed-forward neural network (*FFN*) that processes each position independently to enhance representation and capture complex patterns in the sequence. The self-attention and feed-forward layers are followed by layer normalization, stabilizing, and accelerating training. Additionally, skip connections (*residual*

connections) is applied around the self-attention and feed-forward sub-layers, allowing the model to retain information from previous layers and help gradients flow through the network during backpropagation. This combination of self-attention, feed-forward networks, layer normalization, and skip connections enables the encoder to build highly contextualized representations of the input text.

6.4 Output embeddings

Output embeddings in transformer-based models serve a critical role in mapping the final hidden states of the model to a vocabulary space, enabling the generation of predictions such as words or tokens. Represented as a learned parameter matrix (akin to the Input Embedding), each row of the output embedding [142] corresponds to a specific word in the vocabulary, facilitating the conversion of the model's output vectors into logits. The output vectors are multiplied by the output embedding matrix to produce logits, done through a dot product operation Eq. 7.

$$\text{logits} = \text{output vectors} \times \text{output embeddings}^T \tag{7}$$

where T: Transposed output embedding matrix

These logits represent unnormalized scores for each vocabulary token. Subsequently, the logits undergo a softmax transformation to produce a probability distribution over the vocabulary, allowing the model to predict the next word in a sequence. Output embeddings are designed to map the model's internal representations back to actual tokens for coherent text generation. Their effective training is essential for improving the model's predictive accuracy and ensuring contextually relevant outputs.

6.5 Decoder

In a transformer architecture, the decoder processes positionally encoded embeddings from both input and output sequences. Positional encoding plays a crucial role, allowing the model to understand the correct order of tokens in the sequences. This positioning information enables the decoder to accurately represent the sequences' structure. The attention mechanism within the decoder leverages contextual information from the encoder, significantly enhancing the quality of the generated output. The primary function of the decoder is to generate output sequences from the encoded input sequences, resulting in a series of tokens, often corresponding to words or sub-words. In transformer models like GPT, the model functions as a decoder, using a masked self-attention mechanism to process the input sequence without relying on an explicit encoder by [143]. The decoder is a transformer model that processes input sequences to generate output sequences by leveraging embeddings, positional encoding, and attention mechanisms. The self-attention mechanism in the decoder computes scores as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right)V \tag{8}$$

Where

- $QK^T/\sqrt{d_k}$ scales the dot product attention by the dimension of the key vectors d_k.
- M is a mask matrix preventing future token attention. This structure enables the decoder to generate coherent output sequences based on the contextual information provided by the encoder.

6.6 Encoder–decoder

The purpose of sequence-to-sequence models is to handle tasks where input sequences are mapped to output sequences. These models integrate encoders and decoders. According to these models, the encoder's attention layers have access to every token in the input sequence at every layer. Still, the decoder's attention levels can only access the tokens in the input sequence before a specified word. These models undergo pre-training with goals particular to encoder or decoder tasks and frequently entail extra complexity. For example, some models are pre-trained by predicting words disguised inside text spans. Encoder-decoder models are very helpful in applications where the objective is to generate new text based on an input sequence, such as summarization, translation, and generative question answering.

6.7 Linear layer and softmax

The fully connected linear layer is an essential component of the neural network. Creates a higher-dimensional space from the output embeddings. This transformation is required to transfer the output embeddings back to the original input space to improve the expressiveness of the representation. This allows the model to identify more intricate patterns and correlations in the data. After the decoder stack generates a contextualized output for a specific position, this output is processed through a linear layer, followed by a softmax layer to predict the next word in the sequence. The softmax function generates a probability distribution across the vocabulary for each output word, providing probabilistic output tokens [144]. This architecture showcases the flow of features through the linear layer and the subsequent application of the softmax activation function to yield accurate output probabilities. The logits are calculated as:

$$\text{logits} = W \cdot h + b \tag{9}$$

where W is the weight matrix, h is the decoder output vector, and b is the bias vector. The softmax function converts these logits into a probability distribution:

$$\text{log prob}_i = \frac{e^{\text{logits}_i}}{\sum_{j=1}^{\text{vocab size}} e^{\text{logits}_j}} \tag{10}$$

Finally, the next word is selected using the argmax operation:

$$\text{next word index} = \arg\max_i (\text{log prob}_i) \tag{11}$$

This enables the model to predict the most probable next word by maximizing the probability of the next word through an argmax operation, as shown in Eq. (11). While this approach is practical, there are alternative strategies to select the next word that may be better suited for specific tasks or applications. For example, instead of choosing the word with the highest probability, **sampling** can introduce prediction variability. In sampling, the next word is drawn from the softmax-generated probability distribution, allowing the model to explore less probable but potentially valid sequences. The process is defined as:

$$w_t \sim P(w_t | w_{<t}), \tag{12}$$

where $P(w_t | w_{<t})$ represents the probability distribution for the next word given the previous sequence.

Another approach is **top-k sampling**, where the model restricts its choices to the k most probable words and then samples from this reduced set. This is mathematically described

by retaining the k-highest probabilities from the softmax output, normalizing them, and sampling the next word as:

$$\hat{P}(w_t|w_{<t}) = \frac{P(w_t|w_{<t})}{\sum_{w_{t,i} \in S_k} P(w_{t,i}|w_{<t})}, \tag{13}$$

where S_k is the set of the top-k candidates.

Additionally, **diverse beam search** is another advanced technique. Unlike traditional beam search, which selects hypotheses solely based on log probabilities, diverse beam search introduces penalties for overlapping candidates among different groups in the beam [145]. The adjusted score for candidate w_t is:

$$s'(w_t) = s(w_t) - \lambda \cdot \text{similarity}(w_t, \text{other groups}), \tag{14}$$

where $s(w_t)$ is the log probability score, λ controls the diversity penalty, and similarity measures overlap with other hypotheses. The model can balance precision, diversity, and creativity in generating sequences by incorporating these alternative next-word selection strategies. These methods are instrumental in tasks requiring exploration or diverse outputs, such as dialog systems or creative text generation. Including a discussion on these strategies broadens the scope and utility of the presented framework, demonstrating its flexibility across various use cases.

6.8 Low-rank adaptation

Low-rank adaptation (LoRA) was developed to efficiently fine-tune LLMs while reducing the computational and memory requirements typically associated with full model retraining. Each layer of the model has trainable low-rank matrices. To help achieve this [146] employ a limited collection of trainable parameters, known as adapters, to reduce the required memory without altering the entire set of fixed model parameters. During SGD, gradients pass through the fixed pre-trained model weights as shown in Fig. 13 and are then received by the adaptor, which is adjusted to optimize the loss function. LoRA enhances this process by introducing a factorized projection to improve a linear projection. Equations (15) and (16) are used to compute LoRA as follows.

$$XW = Y \quad \text{with} \quad X \in \mathbb{R}^{b \times h},\ W \in \mathbb{R}^{h \times o} \tag{15}$$

$$Y = XW + sXL1L2 \tag{16}$$

where $L1 \in \mathbb{R}^{h \times r}$, $L2 \in \mathbb{R}^{r \times o}$, and s is a scalar.

One large-scale, pre-trained LM must be modified for several downstream applications to be used in many NLP applications. Usually, fine-tuning, which modifies every parameter of the trained model, is used to achieve this adaptability. The main drawback of fine-tuning is that the new model has parameters different from those of the old model. This poses only a minor inconvenience for fine-tuning. Models like GPT-2 or RoBERTa have significantly fewer trainable parameters compared to larger models like GPT-3, which has 175 billion trainable parameters [19, 77, 81]. New models with even larger parameter counts are being released every few months.

LLMs in AI have grown significantly in size and capability in recent years. Despite the impressive efficacy of these models in a range of NLP tasks, their growing size has caused deployment issues and sparked worries about excessive energy consumption and possible adverse effects on the environment and economy [147, 148]. Using post-training quantization to build low-bit inference models is one way to overcome these difficulties [149]. Reducing

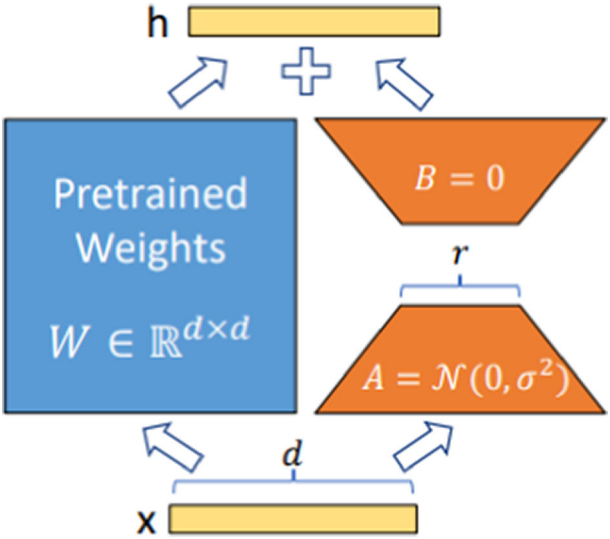

Fig. 13 An illustration of LoRA reparametrization [146]

the accuracy of weights and activations significantly lowers LLMs' memory and processing needs. Smaller bit sizes, including 4-bit versions, are now more popular than 16-bit versions [150]. Post-training quantization is often used in industrial LLMs; however, it is not always the best option.

1-Bit model architectures have been investigated in recent publications, such as [151], as a potential means of lowering the costs of LLMs without sacrificing performance. Matrix multiplication is the main computing activity for conventional LLMs, which commonly utilize 16-bit floating-point formats, such as FP16 or BF16. Consequently, the floating-point addition and multiplication operations account for most of the calculation cost. By comparison, BitNet's matrix multiplication requires only integer addition, saving LLMs orders of magnitude in energy costs. Energy reductions can also be converted into quicker computing, as power is often the primary limiting factor for computing performance in many processors. Transferring model parameters during inference and computation from DRAM to on-chip accelerator memory may be expensive, like SRAM. Though this results in far greater prices than DRAM, attempts have been made to increase the size of SRAM to increase throughput. The capacity and bandwidth of 1-bit LLMs are significantly smaller than those of full-precision variants. Faster and more effective inference can result from this substantial reduction in the cost and time required to load weights from DRAM.

6.9 Quantized low-rank adaptation finetuning

Quantized Low-Rank Adaptation (*QLoRA*) is an advanced technique designed to optimize the performance of LLMs by converting them to a lower memory format, which helps modify undesired behaviors [152]. But large-scale model fine-tuning is still unaffordable [25, 153] report that fine-tuning a LLaMA model with 65 billion parameters necessitates about 780 GB of GPU RAM or almost 16 times more than typical. Although recent quantization techniques can lower LLMs' memory footprints [154], they perform poorly during training and are only helpful for inference [155]. It is demonstrated that, for the first time, a quantized 4-bit model

can be fine-tuned without any loss in performance. According to [146], In QLoRA, a small set of learnable low-rank adapter weights is introduced. These weights are quantized and then updated through the backpropagation of gradients. A novel high-precision method quantizes the model to a 4-bit representation.

QLORA offers several advances that lower memory usage without compromising functionality, including:

1. 4-bit Normal Float: Information-theoretically ideal for quantizing customarily distributed data, this data type generates superior empirical results than 4-bit integers and 4-bit floats.
2. Double Quantization: By quantizing the quantization constants, this method saves, on average, 0.37 bits per parameter (*or around 3 GB for a 65 billion parameter model*).
3. Paged Optimizers: Using Nvidia unified memory, these optimizers avoid memory spikes while processing mini-batches with lengthy sequence lengths during gradient checkpointing.

Incorporating these developments into an enhanced LoRA technique almost eliminates the accuracy trade-offs associated with using adapters at each network layer. To achieve high-fidelity 4-bit finetuning, QLORA introduces two methods: Double Quantization and 4-bit Normal Float (*NF4*) quantization. Furthermore, memory spikes during gradient checkpointing are mitigated by introducing Paged Optimizers, which have previously resulted in out-of-memory failures and complicated large-scale model finetuning on a single machine. QLORA provides a single computation data type (*usually BFloat16*) and a single low-precision storage data type (*standard 4-bit*). 16-Bit matrix multiplication is always preceded by the dequantization of the QLORA weight tensor to BFloat16.

7 Applications of LLMs

This section explores the application of LLMs across various domains, including medicine, chatbots, education, finance, journalism, law, and engineering. The practitioner would like to be provided with an exact understanding of where all these technologies are applied in practice. Some of their common uses are also indicated. The reason for choosing to present fields here is that they are very important and relevant and can potentially make a big impact. These examples show how flexible and powerful LLMs can be in solving complex problems and helping with user tasks [156].

7.1 Health care

It is difficult for healthcare workers to keep current because of the abundance of medical literature and data. Quick analysis of this data by LLMs yields insightful conclusions and suggestions for patient management. Analysis of the Impact of using Health care domain of LLMs

- *Benefits of this domain*: Enhances individualized treatment programs and diagnostic precision.
- *Negative impact*: There is a chance that data biases will result in inaccurate suggestions.

In healthcare, LLMs offer significant potential to revolutionize various aspects. Models like ChatGPT by OpenAI are constructed using extensive training data to understand and generate language akin to human speech. The healthcare sector generates copious amounts of textual data daily, including clinical notes, research papers, patient records, and medical records,

necessitating powerful NLP tools like LMs. These models efficiently analyze, interpret, and extract valuable insights from abundant data, aiding healthcare professionals in administrative tasks, diagnosis, treatment planning, and medical research. Leveraging LMs, healthcare organizations can improve patient outcomes, streamline workflows, enhance decision-making processes, and advance medical knowledge and practice.

ChatGPT and other LLMs have shown great potential in various healthcare applications [157]. Clinical genetics, radiology decision-making, medical education, and patient care have all benefited from their practical use [158]. ChatGPT has become a popular interactive learning and problem-solving tool in medical education [159]. ChatGPT performed as well as, or better than, passing the United States Medical Licensing Exam (*USMLE*) without needing further training or assistance. Moreover, the explanations provided by ChatGPT came out to be coherent and clear in understanding.

A new benchmark dataset is introduced in the MultiMedQA [160] to evaluate LLMs across clinical tasks, which unifies the following six existing datasets of medical question answering. Besides, it makes up consumer, research, and professional questions. Moreover, it introduced the concept of instruction prompt tuning [161], which has enormous potential to enhance the effectiveness of LLMs in a diverse range of clinical tasks. The paper highlights the potential of ChatGPT to act as an assistant in radiologic decision-making because of the evidence of its effectiveness and advantages to improving clinical workflow, as well as the promotion of responsible utilization of radiology services [162]. Research has shown that LLMs, including ChatGPT, can improve the delivery of personalized, empathetic, and scalable healthcare. Both clinical decision-making and medical education stand to benefit from these models.

Duong and Solomon by [163] found that ChatGPT performed similarly to humans in clinical genetics when answering questions about genetics. The model was more accurate for tasks requiring memory than those needing critical thinking. The study also revealed that ChatGPT provided varied responses when presented with the same topic multiple times, offering logical justifications for correct and incorrect answers [164]. Assessed ChatGPT's performance in addressing resuscitation and life support questions. The results indicated that ChatGPT accurately answered most questions from the American Heart Association's basic and advanced cardiovascular life support examinations.

LLMs have been studied for their role in providing better neurosurgical research and patient care across domains such as patient data gathering, survey and questionnaire implementation, care and treatment education, machine-learning [165], NLP, data mining, and decision support [166]. The authors remark on the need for reproducibility in developing AI models and raise the problem in the above-mentioned research fields. Additionally, AI-driven chatbots can improve patient outcomes by streamlining communication between patients and healthcare professionals. These chatbots can provide patients with readily available information regarding their care and treatment using NLP [167].

Many tools, such as Ada Health [168], Babylon Health [169], and Buoy Health [170], are already in use, enabling systems to communicate with patients. LLMs have gained popularity recently, which could enhance both the services offered and patients' confidence in dealing with these chatbots. Many tools have been developed to help medical professionals. One such product is XrayGPT [171], which allows users or patients to ask questions about the analysis during an automated analysis of X-ray images. A user-to-user interactive chat session provides insight into their condition. Another notable advancement is Meta's Segment Anything (*SAM*) model, which can be adapted for diverse applications in medical imaging; DrugGPT [172] is a technology developed in drug discovery, enabling the generation of potential ligands by prompting users with text to target specific proteins.

7.2 Finance

It is necessary because of this application. Complex data analysis is required for personal money management and the financial markets. LLMs are capable of automating trading methods and providing individualized financial advice.

- *Positive impact of this domain*: Strengthens investing and financial decision-making techniques.
- *Negative impact*: When markets experience high volatility, the complexity of rapidly changing conditions can lead to errors in automated systems, resulting in significant financial losses.

LLMs are significantly advancing the finance sector with applications in finance, risk assessment, algorithmic trading, and market prediction [173–175]. They also enhance financial reporting; notably [176], an LLM with 50 billion parameters trained on a large and diverse financial corpus, has revolutionized financial NLP tasks. It excels in classifying news, identifying entities, and responding to inquiries. By leveraging extensive financial data, BloombergGPT improves customer service by promptly addressing queries and providing personalized financial advice. Moreover, LLMs play a crucial role in risk assessment and management by analyzing past market trends and data. They utilize various financial algorithms to recognize potential hazards and propose mitigation strategies. Financial institutions rely on LLMs to make well-informed decisions regarding investment strategies, loan approvals, and credit risk assessment [177].

In the domain of algorithmic trading, LLMs offer substantial benefits. They leverage predictive analytics to identify market opportunities and optimize trading strategies. However, addressing privacy concerns and implementing robust data protection measures to promote LLMs' ethical and compliant application in the finance sector is essential. LLMs serve as predictive and analytical tools, offering the potential to identify market opportunities through algorithmic trading [178]. However, ensuring their use complies with data protection regulations and is both efficient and legal is crucial. Establishing data encryption, redaction, and robust data security processes is imperative since financial information is sensitive and privacy concerns are real. Other projects have been developed to solve these issues, such as FinGPT, an open-source LLM explicitly designed for the banking sector [179]. Through facilitating the appropriate and secure use of LLMs in financial applications, these initiatives seek to address the unique demands and concerns of the banking industry. Further developments in this field are expected to gradually improve the capabilities and uses of LLMs in the financial sector.

7.3 Media and entertainment

It is necessary for this domain. Creating content takes much work. LLMs can effectively generate articles, scripts, and other media material by automating portions of this process.

- *Positive impact of this domain*: It makes content creation more efficient and enables customized media experiences.
- *Negative impact*: A possible reduction in the uniqueness of the material and job displacement for authors and other creators.

With an emphasis on data and customer-centric experiences [180], the media and entertainment industry is transitioning tremendously. All businesses are trying to launch new, creative

products that allow for mass customization of one-to-one communications [181]. The business revolves around content, and as data, particularly social signals, are incorporated into content strategy, the way content is created is also changing. How businesses use data and AI for content generation and LLMs are revolutionizing curation [182]. They help to create attention-grabbing headlines and text, and they offer real-time feedback on material, which helps to improve quality and streamline production processes. LLMs are unique among the many technologies causing this transformation in how they alter the game. Along with their ability to generate unique content, LLMs exhibit a deep understanding of complex data and the capability to simulate human-like interactions. A notable illustration is MediaGPT, a sizable LM explicitly designed for the Chinese media domain [183], capable of generating relevant and high-quality outputs for various applications. LLMs are increasingly applicable in diverse linguistic and cultural contexts, as evidenced by the proposal of Robertuito [184] for Spanish social media.

Large AI models are helpful for various tasks and can generate a wide variety of content, including political speeches, slogans, social media updates, educational videos, and captivating marketing materials [41, 185, 186]. Major entertainment portals provide personalized suggestions for music, TV series, and films using user data analysis and LLM-based algorithms. In addition to increasing customer retention, this improves engagement with entertainment material. Moreover, LLMs are crucial in automated content curation, promoting user retention, satisfaction, and revenue. This trend is exemplified by Dolly, a language model trained by Databricks Inc.[2] specifically for media and entertainment applications. Furthermore, the emergence of AI-powered newscasters [187], such as the virtual newscaster "*Fedha*" [3] introduced by a Kuwaiti media organization in April 2023 underscores the evolving media landscape. Organizations such as the Centre for Journalism at the University of Kent are modifying their programs to prepare future journalists for AI-powered newsrooms.

By leveraging user data, AI systems can provide personalized recommendations for entertainment content, thereby enhancing consumer engagement and retention.[4]

7.4 Marketing

Necessary for this application: Providing individualized and compelling content in a crowded digital market is difficult. Large-scale data analysis by LLMs may help understand customer behavior better and provide exciting content, improving marketing initiatives' efficacy and efficiency.

- *Positive impact of this application*: One advantage is that personalized material increases engagement. It has enhanced productivity through automation of data processing and content production. Better audience segmentation and campaign relevance translate into improved targeting. Savings from automated tasks: Cost reduction.
- *Negative impact*: Repeated, non-unique material might be a risk associated with generic content. Data Privacy Issues: Adherence to data protection measures is crucial. Bias in

[2] Databricks: https://aws.amazon.com/marketplace/pp/prodview-wtyi5lgtce6n6?gclid=EAIaIQobChMIo_6rlIauhwMV6qlmAh1g_QxGEAAYASAAEgLMH_D_BwE&trk=7199a291-54d3-4894-be4d-840d7e08c34d&sc_channel=ps&ef_id=EAIaIQobChMIo_6rlIauhwMV6qlmAh1g_QxGEAAYASAAEgLMH_D_BwE:G:s&s_kwcid=AL!4422!3!670955745549!p!!g!!databricks!20481813768!159548558304.

[3] Skynews: https://news.sky.com/story/google-testing-if-ai-can-write-news-but-insists-it-wont-replace-journalists-12924116.

[4] Aljeeranews:https://news.sky.com/story/google-testing-if-ai-can-write-news-but-insists-it-wont-replace-journalists-12924116.

Data: Marketing material that incorporates biases from training data. Technology Dependence: Danger of marketing losing its human touch.
LLMs are pivotal in contemporary marketing, revolutionizing consumer engagement and information distribution [188] They excel at generating content, saving time, and connecting with audiences [189] by producing exciting blog articles and social media content, ad content, and item descriptions. Personalization enables marketers to send customized communications based on client information, enhancing satisfaction and loyalty [190]. Chatbots are revolutionizing customer care by providing real-time help around the clock and lightening the strain on support staff. LLMs in market research examine enormous amounts of data, including social media and feedback [191], to provide insights into trends, mood, and competitiveness. They uncover keywords, improve social media monitoring, and improve SEO. In addition to the numerous advantages of using LLMs and ChatGPT in marketing, there are some risks that marketers, customers, and other stakeholders may encounter. The consistency and lack of creativity with which ChatGPT responds to identical cues from different marketers might jeopardize the marketer's or brand's unique identity. According to [192], this poses a challenge for marketers who prioritize creativity and innovation in their strategic decision-making, particularly in an AI-driven environment.
AI marketing tools like ChatGPT may provide inaccurate information as they may extract data from questionable sources. An abundance of erroneous data could lead to incorrect results, presenting a worst-case scenario. Additionally, since LLMs can generate human-generated content, ethical considerations arise regarding disclosure and transparency [193]. If AI-generated content [194] overwhelms consumers or is perceived as inauthentic, it may lead to negative consumer perceptions and erode brand trust. Adherence to regulations and data protection laws is essential to mitigate this risk and avoid potential legal consequences. Additionally, marketers should be cautious of becoming overly reliant on external AI suppliers, as this could result in vendor lock-in or the need for alternative platforms.
To overcome these obstacles, marketers need to employ AI sensibly, openly reveal how it is being used, and retain human control to guarantee the appropriateness and precision of material produced by AI. To fit with industry best practices and consumer expectations, marketers employing ChatGPT and LLMs must adhere to ethical rules, conduct frequent audits, engage in continuous monitoring, and analyze and mitigate risks. By carefully balancing the risks and ethical considerations associated with AI adoption, marketers can create opportunities for responsible AI implementation that benefit both the market and consumers.

- *Customer service*: Another sector significantly grasped by LLM technology is customer service. Businesses now have a scalable and effective way to handle client questions and concerns because of the growing integration of virtual assistants and chatbots with LLM skills into customer care systems [195]. Compared to human workers, it improves the overall customer experience [196]. Companies that use LLMs also save money [197]. Since NLP allows for the automatic creation of replies that agents may utilize or amend, many are investigating NLP as a more affordable option than hiring specialized customer care personnel [198]. LLMs appear to be a reasonable and appropriate answer in this situation.
The authors developed a cost-based approach to assess the effectiveness of NLP models in customer support. Three LLM techniques were compared: rapid engineering, fine-tuning, and knowledge distillation informed by agent feedback. Customers could receive assistance from the system through dynamic, context-specific suggestions. This approach

enables the utilization of client-specific data to train and fine-tune ChatGPT, enabling it to deliver unique and tailored responses for each customer. However, despite the promise shown by LLMs and ChatGPT/GPT-4 in customer support, they also present intrinsic drawbacks and challenges. Comprehending complicated enquiries by ChatGPT/GPT-4 and LLMs might challenge context understanding, leading to imprecise and erroneous replies. Moreover, the requirement for human contact, scalability, and emotional intelligence makes delivering flawless customer service interactions much more difficult [199].

7.5 Chatbots

Necessary for this application. In various businesses, chatbots are vital for instant customer assistance and service. They enhance user engagement, assist in handling large client requests, and offer reliable round-the-clock support.

- *Positive impact of this application*: Always available and Provides clients with access at all times, increasing client happiness. Cost-effectiveness: Minimizes the requirement for significant customer support staff. Provides consistent answers to frequently asked questions. Scalability: Able to manage several interactions at once. Data collection: Compiles important client information for enhancement.
- *Negative impact*: Limited Understanding: Difficulty understanding complicated questions or subtle terminology. User Frustration: If needs are not sufficiently addressed, it may irritate users. Updating and maintaining: Requires frequent upgrades. Security concerns: Inadequate security increases the chance of data breaches. Absence of Empathy: Insensitive to human touch, which is essential in delicate circumstances.

In general-purpose chatbots, also called conversation agents, text generation (including code), multi-turn interaction, and IR are amalgamated. The LaMDA family of chatbot LLMs has been introduced by [199], boasting up to 137B parameters. Its primary objectives include factual grounding (*facilitated by access to external knowledge sources*) and safety (*achieved through supervised fine-tuning based on human annotations*). Interestingly, it has been demonstrated that smaller LaMDA models (*2B parameters*) outperform larger LaMDA models (*137B parameters*) without fine-tuning regarding conversation quality and safety/grounding scores when fine-tuned. LaMDA models were integrated into the chatbot Bard as a feature[5] however, the most recent version of Bard currently employs the PaLM 2.

Sparrow is a chatbot based on a 70B parameter Chinchilla Language Model that suggests using RLHF targeting 23 rules to fine-tune the model for enhanced accuracy, helpfulness, and safety [200]. Furthermore, Sparrow uses a retrieval approach to incorporate other knowledge, enabling it to offer proof based on Google Search searches. Regarding output preference and rule violation rate, the RLHF approach performs better than only dialog-prompted and supervised fine-tuning techniques. Similarly, OpenAI used supervised fine-tuning, and RLHF approaches in the ChatGPT chatbot to train a GPT-3.5 LLM for discussed by [201]. The GPT-4 model is the foundation for the ChatGPT Plus chatbot, but the architecture and training data are kept private.

Introducing BlenderBot-3 [202], a chatbot with 175B parameters that use supervised fine-tuning based on the OPT-175 LLM. Through modules that search the internet and extract text-based long-term memories created from prior outputs, BlenderBot-3 integrates external knowledge to support performance across extended interactions. To fine-tune Pythia and

[5] AIJourney: https://blog.google/intl/en-africa/products/explore-get-answers/an-important-next-step-on-our-ai-journey.

LLaMA models (*up to 30B parameters*) for chatbot applications, make available the OpenAssistant Conversations dataset of human-annotated interactions ChatGPT and OpenAI [203]. Guidelines for making the replies courteous, helpful, succinct, pleasant, and safety-aware are incorporated into the dataset to aid in aligning the final models. Currently, the HuggingChat chatbot utilizes the LLaMA 30B version.[6]

Generating a large training dataset of high-quality conversations poses a significant challenge in optimizing chatbots. To address this issue, [204] demonstrates the efficacy of creating synthetic conversation datasets from a limited number of expert-written examples, leveraging the capabilities of LLMs such as the OPT 30B model. Human crowd workers evaluated the generated discussions based on metrics like interestingness, logical coherence, naturalness, and consistency, finding them comparable to previously published human-generated datasets. The utility of these synthetic datasets is showcased in optimizing a chatbot, specifically BlenderBot 400 M, achieving performance levels only marginally lower than those obtained by optimizing with datasets curated by humans.

Evaluating the full spectrum of skills, chatbots exhibit proves incredibly challenging given their universal scope [205]. Testing ChatGPT (*GPT-3.5*) across 25 tasks encompassing 38k prompts, ranging from question answering to emotion identification, spam and offensive language detection, inference, and sentiment analysis, reveals its overall firm performance. However, it frequently lags behind the SOTA in certain domains [206]. In a recent study, OpenAI delves into the potential of GPT-4, the foundational model of ChatGPT Plus, across various contexts, including interactions with both tools and individuals; these assessments showcase GPT-4's remarkable proximity to human-level performance across a diverse array of activities [207].

Ultimately, as LLMs scale up, the issue of inference latency could emerge as a notable hurdle for chatbot applications. A trade-off exists between employing larger LLMs and maintaining real-time, interactive user engagement.[7]

8 Detection of AI-generated text

Identifying whether a person or an LLM wrote a text is called AI-Generated Text Detection (*AGT*), sometimes called Machine Translation Detection (*MTD*). With the advancement of sophisticated LMs capable of generating writing resembling that of humans, this field of study is becoming increasingly essential. Popular LLMs and Tools for Detecting AGT are shown in Fig. 14. The digital ecosystem would only collapse with AGT detection systems, which maintain a lid on online automated content distribution. These systems use cutting-edge ML and NLP to recognize and classify data generated by AI models such as GPT-3, GPT-4, and others. Stopping the spread of false information, safeguarding online communities, and spotting and eliminating fake news are essential to moderation. These technologies allow platform administrators and content moderators to identify AI-generated literature by looking for trends, anomalies in language, and other telltale signs. Despite advances in AI, the need for AGT detection technologies for user security, moral online discourse, and legal online content has remained the same [208].

Nowadays, high-quality texts with numerous applications can be generated via LLMs. One example of a modern ChatGPT developed by [209] model's versatility is its ability to generate human-like narratives for various activities, including composing computer program

[6] Hugging Face: https://huggingface.co/chat/.

[7] OpenAI: https://community.openai.com/t/chat-gpt-4-painfully-slow/117996.

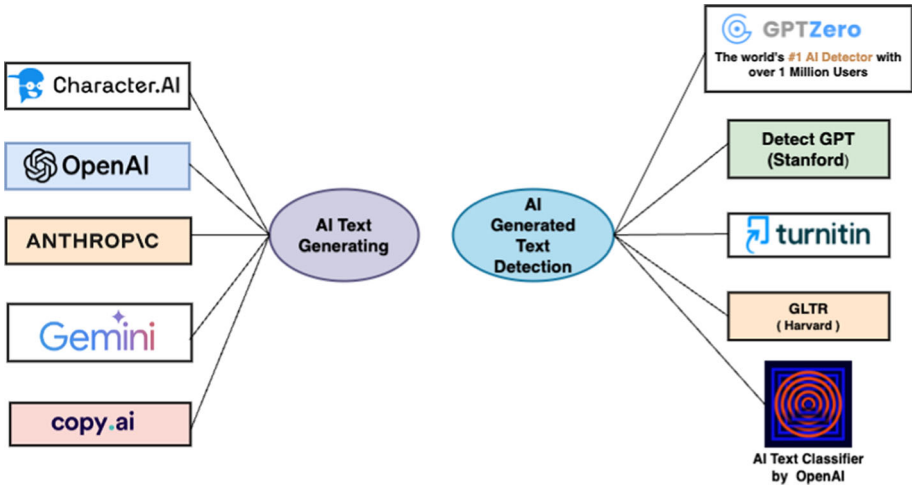

Fig. 14 Popular LLMs and tools for detecting AI-generated text [215]

code, music lyrics, completing documents, and answering questions. These LLMs will only improve with time, according to the NLP trend. However, regulating authenticity and other issues poses a significant challenge. Users may misuse AI systems for unethical activities such as spamming, creating fraudulent product reviews, plagiarism, generating fake news, and manipulating social engineering, all of which could harm society. Some news stories have been corrected by AI, highlighting the importance of ensuring the responsible use of these generative AI techniques [210]. Current research is increasingly focused on identifying AGTs to support responsible usage.

Numerous detection investigations pose the issue as a binary classification challenge and use neural network-based detectors [211]. For instance, to differentiate texts produced by GPT-2 from those authored by humans, OpenAI improves GPT-2 detector models utilizing RoBERTa-based techniques [81]. To guarantee accurate detection, these detectors must be adjusted for every new LLM [212]. Zero-shot AI text identification is the subject of more research since it eliminates the need for extra training overhead. These endeavors assess the textual per-token log-likelihood and utilize thresholding techniques to discern information produced by AI [213]. It is observed that text log probabilities are generally lower for passages created by AI. Detect-GPT is a zero-shot method for detecting text generated by LLMs that leverages the observation that AI-generated content tends to have lower text log probabilities. However, neural network-based detection systems are susceptible to adversarial and poisoning attacks, as discussed by [214].

Another approach to make AGTs more identifiable is through watermarking [182], which leaves specific patterns on the text. Soft watermarking is suggested, categorizing tokens into '*green*' and '*red*' lists and using a pseudo-random generator to select tokens with a high probability from the green list. Although these watermarks may be invisible to humans [216], watermarking detectors can detect them. Additionally, some researchers propose using a database to store LLM outputs for IR-based detection. However, recording user-LLM communications for this purpose raises significant privacy concerns. However, there are two main issues with LLM-created content. First of all, LLMs have the potential to misinterpret signals, fabricate events, use old data, and promote copying, which might reduce accuracy

and contribute to the spread of false information. Second, by mishandling LLMs, users could purposefully provide incorrect information [217].

8.1 AI-generated text detection challenges

NLP has dramatically improved as a result of the rapid development of AI. Consequently, incredibly intricate LMs have been constructed to generate writing resembling a person's. However, these advancements have introduced additional challenges in distinguishing between information generated by machines and information authored by humans, necessitating the creation of reliable and efficient detection systems.

Prompt-based learning centers on estimating the likelihood of a given text excerpt directly, as opposed to traditional supervised ML models, which predict the probability of y based on x [19]. Eliminating specific textual passages and repeatedly retraining the model with the modified portions provides several advantages. Additionally, much textual data is readily available online [77], unlike other models' sparse usage of labeled factual data. In addition, researchers are looking at a different approach to classic NLP in which the analyst's domain expertise and feature engineering determine how good a model is.

By taking away the layer of complexity related to domain knowledge and feature engineering [76], this divergence allows the model to be analyst-agnostic and learn directly from the vast amount of accessible text while allowing for further refinement [80]. Initially, the field relied on fully supervised models to address issues such as sentence length, word recognition, and POS tagging [218]. These models prioritized feature engineering over precise predictions.

Developing convolutional, recurrent, and self-attentional neural networks [18] was accelerated by introducing neural networks. With models tasked with predicting the probability of observed data, the focus shifted back to pre-training and fine-tuning in 2017. Then, tuning improves future sentence predictions [80] and reduces noise by eliminating significant phrases, mainly through MLM [19]. Prompts assisted training tasks by 2021 when the pre-train, prompt, and forecast stages emerged to aid the model's learning from the data. The model is designed to complete partial prompts, and practical, prompt engineering is crucial for improving performance. The authors categorize prompts into two main types: cloze and prefix. Furthermore, prompts can be classified as either discrete or continuous. Discrete prompts, or challenging prompts, are composed of natural language words. In contrast, continuous or soft prompts are embedded directly within the model's framework.

The authors review several response engineering techniques, such as prompt tactics and response space design approaches. Both restricted and unconstrained contexts can benefit from manual design, with the latter frequently being connected to problems involving plenty of options. Pruning-then-search, label decomposition, and paraphrasing are methods for discrete answer search [19]. In this context, training approaches include tuning-free prompting, used in models like GPT-3 [19], and prompt-based and language model fine-tuning techniques, as used in models such as PADA and BERT [80].

Pre-train, prompt, and forecast are typically used in knowledge-probing approaches like linguistic probing and semantic parsing. Information extraction and text categorization are two additional classification tasks they can handle. In the business realm, common sense thinking is a hotly debated issue. Rapid response engineering requires much work, especially when handling long responses, multiple-choice answers, or multiple-class classification problems. The complexities of template and answer are among the other tasks besides categorization and creation.

8.2 Text detection techniques: transformers and decoding approaches

The transformer architecture, renowned for its ability to handle long-term dependencies in text, often serves as the backbone for contemporary LMs. Unidirectional left-to-right approaches have demonstrated their effectiveness in producing coherent language [219]. This method involves the model decoding each sentence using context to determine the probability distribution of different lexical elements [220]. Sometimes, decoding algorithms generate more general language by randomly selecting tokens derived from the output distribution. This leads to text that is challenging for users to distinguish between text that has been synthesized and text that humans have created [221]. ML algorithms, however, are better equipped to identify these patterns. A text becomes more accessible for people to understand as it uses more unusual terms.

8.3 Text detection using machine learning

Numerous ML models have been developed to detect and identify AGT. Prominent models include refined BERT, Bag-of-Words, Histogram-of-Likelihood Ranks, and total probability [222]. These models have been fine-tuned to improve detection accuracy and adapt to evolving AGT techniques. Logistic regression (*LR*), utilizing Term Frequency-Inverse Document Frequency (*TF-IDF*) features, is also employed for this purpose [223]. Another approach involves Support Vector Machines (*SVMs*), which analyze textually, including features such as grammar and semantics. Although SVM classifiers are relatively straightforward to implement and require minimal pre-training, they require extensive re-training when adjustments are made by [224].

The distinction between AGT and Human-written text (*HWT*) has become a crucial area of research because of notable advancements in NLG models like GPT-3. The ability to differentiate between content generated by humans and AI is essential for many uses, such as spam identification, detecting fake news, and ensuring the reliability of internet sources. To get around this problem, ML techniques have been applied to identify subtle characteristics and patterns that set generated text apart from HWT. Advanced LMs and defensive strategies produce false news that may adversely affect [225]. Therefore, one should not undervalue the need to develop robust detection techniques to protect the integrity of online data. The authors suggest funding research to learn more about the dangers and biases present in SOTA LMs.

With a focus on text produced by OpenAI's GPT-3 [223], several researchers examined how well-suited the current detection techniques were for identifying artificially created language. They found that the performance of the existing models in identifying text written by machines varied, with some models doing well and others not performing well. A technique presented in 2020 uses "*BERTscore*" [223] to identify text created by machines. A metric for evaluating produced and reference text semantic similarity is called BERTscore, which uses BERT, a robust language representation model. The study authors found that classifiers based on BERT scores could determine with a high degree of detection accuracy whether the text was authored by a computer or by humans.

9 Context of Indian languages in LLMs

Over a billion people speak the 22 scheduled languages that comprise India's wide linguistic variety.[8] Indian Languages (*ILs*) face challenges in NLP due to their poor resource availability despite their substantial user population. Approximately 234 million individuals can access IL information online [226]. LLMs are transformational instruments that hold great promise for the Indian setting, especially in light of the nation's distinct sociocultural requirements and linguistic variety. Here is a thorough summary of the state of the field's research and present landscape.

LLMs, like the GPT series from OpenAI, have transformed several industries by making it possible to automate processes related to NLP, including translation and text generation. These models have many applications in India, including content creation and translation services. Nonetheless, there are several activities and problems unique to the Indian setting [227]. India has 22+ official languages, including English. People in some of the ILs among the top spoken are native speakers of India. Hindi has 528 million native speakers, Bengali 97.2 million, Marathi 83 million, Telugu 81.1 million, Tamil 69.9 million, Punjabi 33 million, and English 259,678.

LLMs have attracted much attention because of their potential benefits and applicability in various fields.

Because LLMs like OpenAI's GPT series allow automation of NLP activities like translation and text generation, they have revolutionized several sectors. Given India's distinctive social and linguistic Diversity, these models have much potential. Creating specialized models for ILs, such as IndicBERT and MuRIL, demonstrates ongoing attempts to tackle these issues. To further assist NLP research and applications for ILs, tools such as the Indic NLP Library and diverse datasets such as speech recognition and sentiment analysis are being produced (*Analyses India Magazine*). Ultimately, this analysis will benefit many domains, like education, healthcare, and government, by highlighting LLMs' enormous potential and essential problems in improving NLP skills for ILs.

9.1 Challenges within Indian culture

- *Linguistic diversity*: India has hundreds of dialects and 22 officially recognized languages. Developing LLMs that can generate and comprehend text in various languages is challenging. Ensuring cultural sensitivity and accuracy in translations and content creation is essential.[9] Resource Availability: One major problem is to create and maintain comprehensive datasets for all ILs. The large-scale digital text required for training efficient LLMs is absent from several ILs. Possible use cases are mentioned below.
- *Governance*: LLMs can play a critical role in governance by enabling multilingual communication between public authorities and the general public. Automated translation and answer creation can potentially improve accessibility to government services.
- *Education*: To ensure accessibility and diversity, LLMs can help create instructional materials in several languages [228].
- *Healthcare*: When patients speak various languages from healthcare practitioners, multilingual LLMs can assist in closing the communication gap [229].
  LLMs have attracted much attention because of their potential benefits and applicability in various fields.

[8] India's languages: https://indiaai.gov.in/.

[9] MultilingualAI https://www.ruder.io/state-of-multilingual-ai/.

Because LLMs like OpenAI's GPT series allow automation of NLP activities like translation and text generation, they have revolutionized several sectors. Given India's distinctive social and linguistic variety, these models have much potential. Creating specialized models for ILs, such as IndicBERT and MuRIL, demonstrates ongoing attempts to tackle these issues. To further assist NLP research and applications for ILs, tools such as the Indic NLP Library and diverse datasets such as speech recognition and sentiment analysis are being produced (Analyses India Magazine). Ultimately, this analysis will benefit many domains, like education, healthcare, and government, by highlighting LLMs' enormous potential and essential problems in improving NLP skills for ILs.

10 Pros of large language models

- *Increased efficiency and productivity*: LLMs can significantly increase marketing, finance, and customer service productivity by automating and streamlining processes like sentiment analysis, text completion, and language translation. Using NLP, these systems can comprehend spoken and written language similarly to humans. Businesses used ML techniques to train systems to comprehend human writing before introducing LLMs. This procedure has been made simpler with the advent of LLMs, such as GPT-3.5, which has improved the speed and precision of AI-powered devices in comprehending human language. Advanced LLMs like ChatGPT and BARD are examples.
- *Openness to information and information democracy*: Similar to the effects of the internet and search engines, LLMs increase information accessibility by reducing complicated texts and translating languages.
- *Adaptable uses*: These models find use in many contexts, including content generation, virtual tutoring, automated grading in the classroom, and targeted advertising. They do exceptionally well in NLP tasks like opinion mining and IR. Numerous NLP tasks, such as text creation, summarization, translation, question answering, and many more, may be completed by LLMs.
- *Human-like interaction*: LLMs are ideal for developing complex chatbots and virtual assistants that enhance customer support and other interactive app user experiences because they can mimic human speech.

11 Limitations of large language models

Despite their considerable impact on NLP, LLMs have notable limitations.

- *Hallucinations*: Three primary risks associated with using LLMs are hallucination, bias, and deception. Since LLMs are trained to predict the next word in a sequence, they do not always generate accurate outputs. The occurrence of LLMs producing factually incorrect information is known as hallucination. The phenomenon of "*hallucinations*" in the context of LMs has garnered significant interest. Hallucination in LLMs is defined as *"the generation of content that is nonsensical or unfaithful to the provided source"*, as discussed by [103]. Although the term originates from psychological terminology, it has been adapted to describe a specific issue within AI.
  Two general categories can be used to describe hallucinations in LLMs:
  - *Intrinsic hallucinations*: Introduce logical or factual errors directly opposing the original content.

  - *Extrinsic hallucinations*: These include speculative or unconfirmable aspects and, although they do not contradict, cannot be verified against the source. In LLM situations, the meaning of "*source*" changes depending on the purpose. It relates to "*world knowledge*" in dialog-based activities, while text summarizing refers to the input text itself. This difference is essential for assessing and understanding hallucinations. Additionally, hallucination's effects vary greatly depending on the situation. For example, hallucinations may be considered normal and helpful in creative pursuits like poetry writing.

  LLMs generate text using probabilistic models and lack an intrinsic understanding of truth or falsehood, as they are trained on varied sources, including the internet, books, and Wikipedia. Although recent advancements like RLHF and instruction tuning aim to guide LLMs toward generating more accurate outputs, these models' inherent probabilistic nature and limitations remain [230] identifying two significant factors leading to hallucinations in LLMs: the frequency heuristic and the truthfulness bias. This study highlights the complexity of training LLMs and the challenges in generating accurate outputs.

- *Garbage input–output*: One primary concern is the "Garbage In, Garbage Out" (*GIGO*) problem, which states that the quality of the input data significantly impacts the caliber of the output produced by these models. In instances where the training data is contaminated by racist, sexist, or discriminatory information, the model may provide problematic outputs. Below is a synopsis of the critical issues and pertinent studies.

  - *Sexist*: LLMs can potentially reflect and exacerbate social biases in the training data. Research has shown that these models may generate content that is sexist, racist, or otherwise discriminatory [231]. Discuss how these biases manifest in NLP models and explore the consequences of such biases.
  - *Racist*: Because LLMs may create material that seems reasonable but is inaccurate or deceptive, they can create and disseminate misinformation and disinformation [107], the ethical and social risks of LLMs, including their propensity to reproduce harmful biases.
  - *Toxic*: Harmful text, such as aggressive rhetoric, hate speech, and other poisonous words, can be produced by LLMs [232]. Although LLMs can potentially revolutionize various fields, their disadvantages must be carefully considered and mitigated, particularly toxicity, prejudice, disinformation, and ethical issues. Research is ongoing to solve these problems, but there are still many obstacles to overcome before these models can be widely used in a safe, equitable, and dependable way.

- *Memorization*: GPT-4 and other LLMs have substantial limitations and constraints in recalling portions of their training data. Memorization in LLMs may reduce the models' capacity to generalize, cause private and ethical issues, and result in disclosing confidential or proprietary information [233].

  - *Leakage of private information*: It is possible for LLMs to inadvertently learn and replicate private or sensitive information found in training data. This can contain proprietary information, private communications, and personal data [234].
  - *Diminished broadness capacity*: Overfitting can result from memorization's harmful effects on LLM's capacity to generalize from their training set. As a result, they are less able to produce precise and pertinent answers to fresh, hidden suggestions.
  - *Privacy and ethical issues*: When LLMs unintentionally reveal private or sensitive information (PII), memorization presents serious ethical and privacy issues. Memo-

rization in LLMs has several disadvantages, primarily regarding security, data privacy, and these models' capacity for generalization. Robust procedures must be implemented with continuous research to address these problems and reduce the hazards of memorizing. Balancing model performance and privacy protection is crucial when developing and implementing LLMs.
  - *Lack of common sense*: LLMs are adept at comprehending and producing writing that resembles a human's. They frequently lack common sense thinking, which restricts their applicability and dependability in some situations.
  - *A lack of logic based on common sense*: LLMs can generate cohesive and contextually relevant content, though they frequently lack knowledge and common sense thinking. This results from their training on massive text datasets and reliance on statistical patterns over proper understanding [235]. Limitation Examples:
  - *Variability in responses:* LLMs may produce erratic or contradictory answers under different circumstances when asked the same question.
  - *Lack of context understanding:* LLMs might struggle with understanding implicit context or making logical inferences that are evident to humans.
  - *Limited real-world application:* LLMs may fall short when required to understand social or physical dynamics or apply real-world knowledge effectively.

- *Security risks*: LLMs can be deliberately used to create phishing emails that look real or to provide false information. To reduce these threats, strong security measures are required.
- *Job displacement* Certain job areas, such as writing, customer service, and programming, are at risk because of the automated potential of LLMs. This gives rise to worries about the loss of jobs and the requirement for new skill sets in the rapidly changing digital economy.
- *Stochasticity*: Stochasticity refers to their inherent unpredictability in the context of LLMs. This unpredictability arises from the probabilistic methods LLMs use to generate text, where the next word in a sequence is selected according to learned probability distributions. While this stochastic nature allows for diverse and creative text generation, it also presents several challenges:
  - *Inconsistent responses:* LLMs may generate different responses to the same prompt in successive interactions due to stochasticity. This variability poses problems for applications requiring consistent and reliable outputs, such as legal advice, medical consultations, and customer service.
  - *Variable quality:* The quality of responses from LLMs can vary significantly because of their stochastic nature. At times, the generated text may be highly coherent and contextually appropriate, while at other times, it may be nonsensical. This variability complicates using LLMs in professional and mission-critical applications [236].
  - *Challenges in fine-tuning:* Minimizing stochasticity while maintaining creativity and adaptability presents a challenge during fine-tuning. Excessive fine-tuning may lead to overfitting, where the model becomes too rigid and loses its ability to generalize to new inputs effectively [237].

12 Conclusion

LLMs have been thoroughly surveyed in this work, which also looks at their creation, assessment, uses, and underlying technical ideas. Several LLM applications have been examined and discussed, including benefits and drawbacks. This study aims to improve the knowledge and use of these models among the scientific community. By examining various LLM designs, pre-training strategies, capacity assessments, and fine-tuning procedures, an understanding of the essential elements contributing to LLM's performance has been offered. The study also emphasizes the importance of using AI tools like ChatGPT ethically and responsibly. It emphasizes how important it is to create rules and regulations to deal with the difficulties this technology brings. Ensuring LLMs are responsibly integrated across numerous fields, including healthcare, education, chatbots, and business, is crucial. By doing this, these tools may respect the values of justice, honesty, and privacy while assisting and enhancing human endeavors. Future research should address LLM limitations related to interpretability, bias, and data dependency and enhance their precision and adaptability to specialized domains. Progress in multi-modal models, hybrid neural-symbolic systems, and sustainable practices offers promising avenues to improve LLM performance and efficiency. Additionally, collaborative efforts among technologists, ethicists, and policymakers will be essential in establishing guidelines that support ethical, socially beneficial applications. These recommendations may help the research community, which can drive advancements in LLM technologies, ensuring that these tools are both practical and responsibly applied for the benefit of society.

Acknowledgements We are grateful to the National Institute of Technology Silchar's Computer Science & Engineering Department for allowing us to conduct our studies and tests. We also thank the Center for Natural Language Processing, the AI laboratories, and the supportive research environment for their assistance and resources.

Data Availability Data sharing is not applicable to this article as no datasets were generated or analyzed during the current study.

Declarations

Conflict of interest The authors declare that they have no conflict of interest.

References

1. Baud R (1998) Present and future trends with NLP. Int J Med Inform 52(1–3):133–139
2. Simon HA (1995) Artificial intelligence: an empirical science. Artif Intell 77(1):95–127
3. Khurana D, Koli A, Khatter K, Singh S (2023) Natural language processing: state of the art, current trends and challenges. Multimed Tools Appl 82(3):3713–3744
4. Pinker S (2003) The language instinct: how the mind creates language. Penguin UK, London
5. Hauser MD, Chomsky N, Fitch WT (2002) The faculty of language: what is it, who has it, and how did it evolve? Science 298(5598):1569–1579
6. Turing AM (2009) Computing machinery and intelligence. Springer, Berlin
7. Rosenfeld R (2000) Two decades of statistical language modeling: where do we go from here? Proc IEEE 88(8):1270–1278
8. Andreas J, Vlachos A, Clark S (2013) Semantic parsing as machine translation. In: Proceedings of the 51st annual meeting of the association for computational linguistics (volume 2: short papers), pp 47–52
9. Pham V, Bluche T, Kermorvant C, Louradour J (2014) Dropout improves recurrent neural networks for handwriting recognition. In: 2014 14th international conference on frontiers in handwriting recognition, pp 285–290. IEEE
10. Htut PM, Cho K, Bowman SR (2018) Grammar induction with neural language models: an unusual replication. arXiv:1808.10000

11. Ponte JM, Croft WB (2017) A language modeling approach to information retrieval. In: ACM SIGIR Forum, vol 51, pp 202–208. ACM New York, NY, USA
12. Kuhn R, De Mori R (1990) A cache-based natural language model for speech recognition. IEEE Trans Pattern Anal Mach Intell 12(6):570–583
13. Iqbal T, Qureshi S (2022) The survey: text generation models in deep learning. J King Saud Univ Comput Inf Sci 34(6):2515–2528
14. Diao S, Xu R, Su H, Jiang Y, Song Y, Zhang T (2021) Taming pre-trained language models with n-gram representations for low-resource domain adaptation. In: Proceedings of the 59th annual meeting of the association for computational linguistics and the 11th international joint conference on natural language processing (volume 1: long papers), pp 3336–3349
15. Misra K, Mahowald K (2024) Language models learn rare phenomena from less rare phenomena: the case of the missing AANNs. arXiv:2403.19827
16. Qing D, Zheng Y, Zhang W, Ren W, Zeng X, Li G (2024) Semi-supervised feature selection with minimal redundancy based on group optimization strategy for multi-label data. Knowl Inf Syst 66:1–38
17. Mishra A, Soni U, Arunkumar A, Huang J, Kwon BC, Bryan C (2023) Promptaid: prompt exploration, perturbation, testing and iteration using visual analytics for large language models. arXiv:2304.01964
18. Vaswani A (2017) Attention is all you need. In: Advances in neural information processing systems
19. Brown TB (2020) Language models are few-shot learners. arXiv:2005.14165
20. Sadasivan VS, Kumar A, Balasubramanian S, Wang W, Feizi S (2023) Can AI-generated text be reliably detected? arXiv:2303.11156
21. Yin W, Kann K, Yu M, Schütze H (2017) Comparative study of CNN and RNN for natural language processing. arXiv:1702.01923
22. Gu A, Dao T (2023) Mamba: linear-time sequence modeling with selective state spaces. arXiv:2312.00752
23. Manning CD (2022) Human language understanding & reasoning. Daedalus 151(2):127–138
24. Srivastava A, Rastogi A, Rao A, Shoeb AAM, Abid A, Fisch A, Brown AR, Santoro A., Gupta A, Garriga-Alonso A, et al (2022) Beyond the imitation game: Quantifying and extrapolating the capabilities of language models. arXiv:2206.04615
25. Touvron H, Lavril T, Izacard G, Martinet X, Lachaux M-A, Lacroix T, Rozière B, Goyal N, Hambro E, Azhar F, et al (2023) Llama: open and efficient foundation language models. arXiv:2302.13971
26. Luitse D, Denkena W (2021) The great transformer: examining the role of large language models in the political economy of AI. Big Data Soc 8(2):20539517211047736
27. Zhao Y, Lin Z, Zhou D, Huang Z, Feng J, Kang B (2023) Bubogpt: enabling visual grounding in multi-modal LLMS. arXiv:2307.08581
28. Huang J, Chang KC-C (2022) Towards reasoning in large language models: a survey. arXiv:2212.10403
29. Pappas N, Meyer T (2012) A survey on language modeling using neural networks
30. Hadi MU, Al Tashi Q, Shah A, Qureshi R, Muneer A, Irfan M, Zafar A, Shaikh MB, Akhtar N, Wu J, et al (2024) Large language models: a comprehensive survey of its applications, challenges, limitations, and future prospects. Authorea Preprints
31. Goodman J (2001) A bit of progress in language modeling. Technical report, Technical Report
32. Kilgarriff A, Grefenstette G (2003) Introduction to the special issue on the web as corpus. Comput Linguist 29(3):333–347
33. Banko M, Brill E (2001) Scaling to very very large corpora for natural language disambiguation. In: Proceedings of the 39th annual meeting of the association for computational linguistics, pp 26–33
34. Halevy A, Norvig P, Pereira F (2009) The unreasonable effectiveness of data. IEEE Intell Syst 24(2):8–12
35. Chen L, Li S, Bai Q, Yang J, Jiang S, Miao Y (2021) Review of image classification algorithms based on convolutional neural networks. Remote Sens 13(22):4712
36. Bahdanau D (2014) Neural machine translation by jointly learning to align and translate. arXiv:1409.0473
37. Hern A (2019) New AI fake text generator may be too dangerous to release, say creators. The Guardian 14:2019
38. Heaven WD (2023) GPT-4 is bigger and better than ChatGPT—but OpenAI won't say why. MIT Technology Review
39. Beltagy I, Lo K, Cohan A (2019) Scibert: a pretrained language model for scientific text. arXiv:1903.10676
40. Sallam M (2023) The utility of chatgpt as an example of large language models in healthcare education, research and practice: systematic review on the future perspectives and potential limitations. MedRxiv, 2023–02
41. Eloundou T, Manning S, Mishkin P, Rock D (2023) Gpts are gpts: an early look at the labor market impact potential of large language models. arXiv:2303.10130

42. Li B, Mellou K, Zhang B, Pathuri J, Menache I (2023) Large language models for supply chain optimization. arXiv:2307.03875
43. Bariah L, Zhao Q, Zou H, Tian Y, Bader F, Debbah M (2023) Large language models for telecom: The next big thing? arXiv:2306.10249
44. Chen M, Tworek J, Jun H, Yuan Q, Pinto HPDO, Kaplan J, Edwards H, Burda Y, Joseph N, Brockman G, et al (2021) Evaluating large language models trained on code. arXiv:2107.03374
45. Sun Z (2023) A short survey of viewing large language models in legal aspect. arXiv:2303.09136
46. Radford A (2018) Improving language understanding by generative pre-training
47. Yang Z (2019) XLNet: generalized autoregressive pretraining for language understanding. arXiv:1906.08237
48. Dale R (2021) Gpt-3: what's it good for? Nat Lang Eng 27(1):113–118
49. Li C, Balmeo HDA (2024) Automatic text labeling method based on large language models. Int J Emerg Technol Adv Appl 1(1)
50. Gao L, Biderman S, Black S, Golding L, Hoppe T, Foster C, Phang J, He H, Thite A, Nabeshima N, et al (2020) The pile: an 800gb dataset of diverse text for language modeling. arXiv:2101.00027
51. Wang S, Sun Y, Xiang Y, Wu Z, Ding S, Gong W, Feng S, Shang J, Zhao Y, Pang C, et al (2021) Ernie 3.0 titan: exploring larger-scale knowledge enhanced pre-training for language understanding and generation. arXiv:2112.12731
52. Askell A, Bai Y, Chen A, Drain D, Ganguli D, Henighan T, Jones A, Joseph N, Mann B, DasSarma N, et al (2021) A general language assistant as a laboratory for alignment. arXiv:2112.00861
53. Hoffmann J, Borgeaud S, Mensch A, Buchatskaya E, Cai T, Rutherford E, Casas DdL, Hendricks LA, Welbl J, Clark A, et al (2022) Training compute-optimal large language models. arXiv:2203.15556
54. Cheng H, Thoppilan R (2022) LaMDA: towards safe. Grounded, and high-quality dialog models for everything, Google AI, vol 3
55. Black S, Biderman S, Hallahan E, Anthony Q, Gao L, Golding L, He H, Leahy C, McDonell K, Phang J, et al (2022) Gpt-neox-20b: an open-source autoregressive language model. arXiv:2204.06745
56. Narang S, Chowdhery A (2022) Pathways language model (palm): scaling to 540 billion parameters for breakthrough performance. In: Google AI Blog
57. Zhang S, Roller S, Goyal N, Artetxe M, Chen M, Chen S, Dewan C, Diab M, Li X, Lin XV, et al (2022) Opt: open pre-trained transformer language models. arXiv:2205.01068
58. Lewkowycz A, Andreassen A, Dohan D, Dyer E, Michalewski H, Ramasesh V, Slone A, Anil C, Schlag I, Gutman-Solo T et al (2022) Solving quantitative reasoning problems with language models. Adv Neural Inf Process Syst 35:3843–3857
59. Taylor R, Kardas M, Cucurull G, Scialom T, Hartshorn A, Saravia E, Poulton A, Kerkez V, Stojnic R (2022) Galactica: a large language model for science
60. Soltan S, Ananthakrishnan S, FitzGerald J, Gupta R, Hamza W, Khan H, Peris C, Rawls S, Rosenbaum A, Rumshisky A, Prakash CS, Sridhar M, Triefenbach F, Verma A, Tur G, Natarajan P (2022) Alexatm 20b: few-shot learning using a large-scale multilingual seq2seq model
61. Schreiner M (2023) Gpt-4 architecture, datasets, costs and more leaked. In: The decoder, vol 11
62. Penedo G, Malartic Q, Hesslow D, Cojocaru R, Cappelli A, Alobeidli H, Pannier B, Almazrouei E, Launay J (2023) The refinedweb dataset for falcon LLM: outperforming curated corpora with web data, and web data only
63. Wu S, Irsoy O, Lu S, Dabravolski V, Dredze M, Gehrmann S, Kambadur P, Rosenberg D, Mann G (2023) Bloomberggpt: a large language model for finance
64. AI M (2023) Introducing LLaMA: a foundational, 65-billion-parameter large language model. Accessed: 30 Aug 2024. https://doi.org/10.48550/arXiv.2302.13971
65. Bubeck S, Chandrasekaran V, Eldan R, Gehrke J, Horvitz E, Kamar E, Lee P, Lee YT, Li Y, Lundberg S, Nori H, Palangi H, Ribeiro MT, Zhang Y (2023) Sparks of artificial general intelligence: early experiments with GPT-4
66. Team L (2024) The llama 3 herd of models. arXiv:2407.21783
67. Manning C, Schutze H (1999) Foundations of statistical natural language processing. MIT Press, Cambridge
68. Hochreiter S (1997) Long short-term memory. In: Neural computation. MIT-Press
69. Hihi S, Bengio Y (1995) Hierarchical recurrent neural networks for long-term dependencies. In: Advances in neural information processing systems, vol 8
70. Zhai C (2008) Statistical language models for information retrieval: a critical review. Found Trends® Inf Retrieval 2(3):137–213 https://doi.org/10.1561/1500000006
71. Katz SM (1987) Estimation of probabilities from sparse data for the language model component of a speech recognizer

72. Gale WA, Sampson G (1995) Good-turing frequency estimation without tears. J. Quant Linguist 2:217–237. https://doi.org/10.1080/09296179508590051
73. Kombrink S, Mikolov T, Karafiát M, Burget L (2011) Recurrent neural network based language modeling in meeting recognition
74. Mikolov T, Karafiát M, Burget L, Cernockỳ J, Khudanpur S (2010) Recurrent neural network based language model. In: Interspeech, vol 2, pp 1045–1048. Makuhari
75. Bengio Y, Ducharme R, Vincent P. A neural probabilistic language model
76. Mikolov T (2013) Efficient estimation of word representations in vector space, vol 3781. arXiv:1301.3781
77. Radford A, Wu J, Child R, Luan D, Amodei D, Sutskever I. Language models are unsupervised multitask learners. https://github.com/codelucas/newspaper
78. Devlin J, Chang M-W, Lee K, Toutanova K (2018) Bert: pre-training of deep bidirectional transformers for language understanding
79. Fedus W, Zoph B, Shazeer N (2022) Switch transformers: scaling to trillion parameter models with simple and efficient sparsity. J Mach Learn Res 23(120):1–39
80. Devlin J (2018) Bert: pre-training of deep bidirectional transformers for language understanding. arXiv:1810.04805
81. Liu Y (2019) Roberta: A robustly optimized bert pretraining approach, vol 364. arXiv:1907.11692
82. Mikolov T, Sutskever I, Chen K, Corrado GS, Dean J (2013) Distributed representations of words and phrases and their compositionality. In: Advances in neural information processing systems, vol 26
83. Pennington J, Socher R, Manning CD (2014) Glove: global vectors for word representation. In: Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP), pp 1532–1543
84. Howard J, Ruder S (2018) Universal language model fine-tuning for text classification. arXiv:1801.06146
85. Pang B, Lee L, et al (2008) Opinion mining and sentiment analysis. Found Trends® Inf Retrieval 2(1–2):1–135
86. Clark K (2019) What does bert look at? An analysis of bert's attention. arXiv:1906.04341
87. Bertolotti F, Cazzola W (2023) CombTransformers: statement-wise transformers for statement-wise representations. IEEE Trans Softw Eng. https://doi.org/10.1109/TSE.2023.3310793
88. Zaheer M, Guruganesh G, Dubey KA, Ainslie J, Alberti C, Ontanon S, Pham P, Ravula A, Wang Q, Yang L et al (2020) Big bird: transformers for longer sequences. Adv Neural Inf Process Syst 33:17283–17297
89. Beltagy I, Peters ME, Cohan A (2020) Longformer: the long-document transformer. arXiv:2004.05150
90. Liu Z, Huang Y, Xia X, Zhang Y (2024) All is attention for multi-label text classification. Knowl Inf Syst. https://doi.org/10.1007/s10115-024-02253-w
91. Raffel C, Shazeer N, Roberts A, Lee K, Narang S, Matena M, Zhou Y, Li W, Liu PJ (2020) Exploring the limits of transfer learning with a unified text-to-text transformer. J Mach Learn Res 21(140):1–67
92. Hernandez D, Brown T, Conerly T, DasSarma N, Drain D, El-Showk S, Elhage N, Hatfield-Dodds Z, Henighan T, Hume T, et al (2022) Scaling laws and interpretability of learning from repeated data. arXiv:2205.10487
93. Ouyang L, Wu J, Jiang X, Almeida D, Wainwright C, Mishkin P, Zhang C, Agarwal S, Slama K, Ray A et al (2022) Training language models to follow instructions with human feedback. Adv Neural Inf Process Syst 35:27730–27744
94. Su J, Ahmed M, Lu Y, Pan S, Bo W, Liu Y (2024) Roformer: enhanced transformer with rotary position embedding. Neurocomputing 568:127063
95. Sutton RS, Barto AG (2018) Reinforcement Learning: An Introduction. MIT Press, Cambridge
96. Mnih V, Kavukcuoglu K, Silver D, Rusu AA, Veness J, Bellemare MG, Graves A, Riedmiller M, Fidjeland AK, Ostrovski G et al (2015) Human-level control through deep reinforcement learning. Nature 518(7540):529–533
97. Mialon G, Dessì R, Lomeli M, Nalmpantis C, Pasunuru R, Raileanu R, Rozière B, Schick T, Dwivedi-Yu J, Celikyilmaz A, Grave E, LeCun Y, Scialom T (2023) Augmented language models: a survey
98. Liu X, Cheng H, He P, Chen W, Wang Y, Poon H, Gao J (2020) Adversarial training for large neural language models. arXiv:2004.08994
99. Chelba C, Mikolov T, Schuster M, Ge Q, Brants T, Koehn P, Robinson T (2013) One billion word benchmark for measuring progress in statistical language modeling. arXiv:1312.3005
100. Biderman S, Schoelkopf H, Sutawika L, Gao L, Tow J, Abbasi B, Aji AF, Ammanamanchi PS, Black S (2024) Lessons from the trenches on reproducible evaluation of language models. arXiv:2405.14782
101. Dai Z (2019) Transformer-xl: Attentive language models beyond a fixed-length context. arXiv:1901.02860
102. Bottou L (2012) Stochastic gradient descent tricks. In: Neural networks: tricks of the trade, 2nd edn, pp 421–436. Springer, Berlin

103. Ji Z, Lee N, Frieske R, Yu T, Su D, Xu Y, Ishii E, Bang YJ, Madotto A, Fung P (2023) Survey of hallucination in natural language generation. ACM Comput Surv 55(12):1–38
104. Bommasani R, Hudson DA, Adeli E, Altman R, Arora S, Arx S, Bernstein MS, Bohg J, Bosselut A, Brunskill E, et al (2021) On the opportunities and risks of foundation models. arXiv:2108.07258
105. McGuffie K, Newhouse A (2020) The radicalization risks of gpt-3 and advanced neural language models. arXiv:2009.06807
106. Wallace E, Feng S, Kandpal N, Gardner M, Singh S (2019) Universal adversarial triggers for attacking and analyzing NLP. arXiv:1908.07125
107. Bender EM, Gebru T, McMillan-Major A, Shmitchell S (2021) On the dangers of stochastic parrots: can language models be too big. In: Proceedings of the 2021 ACM conference on fairness, accountability, and transparency, pp 610–623
108. Solaiman I, Brundage M, Clark J, Askell A, Herbert-Voss A, Wu J, Radford A, Krueger G, Kim JW, Kreps S, et al (2019) Release strategies and the social impacts of language models. arXiv:1908.09203
109. Wu S, Irsoy O, Lu S, Dabravolski V, Dredze M, Gehrmann S, Kambadur P, Rosenberg D, Mann G (2023) Bloomberggpt: a large language model for finance. arXiv:2303.17564
110. Zhu Y (2015) Aligning books and movies: towards story-like visual explanations by watching movies and reading books. arXiv:1506.06724
111. Praveen S, Vajrobol V (2023) Understanding the perceptions of healthcare researchers regarding ChatGPT: a study based on bidirectional encoder representation from transformers (BERT) sentiment analysis and topic modeling. Ann Biomed Eng 51(8):1654–1656
112. Salazar J, Liang D, Nguyen TQ, Kirchhoff K (2019) Masked language model scoring. arXiv:1910.14659
113. Sun Y, Zheng Y, Hao C, Qiu H (2021) Nsp-bert: a prompt-based few-shot learner through an original pre-training task–next sentence prediction. arXiv:2109.03564
114. Zhao W, Hu H, Zhou W, Shi J, Li H (2023) Best: bert pre-training for sign language recognition with coupling tokenization. In: Proceedings of the AAAI conference on artificial intelligence, vol 37, pp 3597–3605
115. Akbar NA, Darmayanti I, Fati SM, Muneer A (2021) Deep learning of a pre-trained language model's joke classifier using GPT-2. J Hunan Univ Nat Sci 48(8):1–7
116. Fan L, Li L, Ma Z, Lee S, Yu H, Hemphill L (2024) A bibliometric review of large language models research from 2017 to 2023. ACM Trans Intell Syst Technol 15(5):1–25
117. Dettmers T, Lewis M, Belkada Y, Zettlemoyer L (2022) Gpt3. int8: 8-bit matrix multiplication for transformers at scale. In: Advances in neural information processing systems, vol 35, pp 30318–30332
118. Su J, Yu S, Luo D (2020) Enhancing aspect-based sentiment analysis with capsule network. IEEE Access 8:100551–100561
119. Kolbæk M, Yu D, Tan Z-H, Jensen J (2017) Multitalker speech separation with utterance-level permutation invariant training of deep recurrent neural networks. IEEE/ACM Trans Audio Speech Lang Process 25(10):1901–1913
120. Hobbhahn M, Lieberum T, Seiler D (2022) Investigating causal understanding in LLMs. In: NeurIPS ML safety workshop
121. Zhao WX, Zhou K, Li J, Tang T, Wang X, Hou Y, Min Y, Zhang B, Zhang J, Dong Z, et al (2023) A survey of large language models. arXiv:2303.18223
122. Shen W, Chen J, Quan X, Xie Z (2021) Dialogxl: All-in-one xlnet for multi-party conversation emotion recognition. In: Proceedings of the AAAI conference on artificial intelligence, vol 35, pp 13789–13797
123. Cho J, Lei J, Tan H, Bansal M (2021) Unifying vision-and-language tasks via text generation. In: Proceedings of the international conference on machine learning, pp 1931–1942. PMLR, Virtual
124. Nandi S, Curado Malta M, Maji G, Dutta A (2024) IC-SNI: measuring nodes' influential capability in complex networks through structural and neighboring information. Knowl Inf Syst. https://doi.org/10.1007/s10115-024-02262-9
125. Luo R, Sun L, Xia Y, Qin T, Zhang S, Poon H, Liu T-Y (2022) Biogpt: generative pre-trained transformer for biomedical text generation and mining. Brief Bioinform 23(6):409
126. Ushio A, Alva-Manchego F, Camacho-Collados J (2022) Generative language models for paragraph-level question generation. arXiv:2210.03992
127. Deutsch D, Juraska J, Finkelstein M, Freitag M (2023) Training and meta-evaluating machine translation evaluation metrics at the paragraph level. arXiv:2308.13506
128. Keskar NS, McCann B, Varshney LR, Xiong C, Socher R (2019) Ctrl: a conditional transformer language model for controllable generation. arXiv:1909.05858
129. Rebuffi S-A, Bilen H, Vedaldi A (2017) Learning multiple visual domains with residual adapters. In: Advances in neural information processing systems, vol 30
130. AI M (2019) Roberta: an optimized method for pretraining self-supervised NLP systems. Technical report, Facebook

131. Wang A (2018) Glue: a multi-task benchmark and analysis platform for natural language understanding. arXiv:1804.07461
132. Tian S, Jin Q, Yeganova L, Lai P-T, Zhu Q, Chen X, Yang Y, Chen Q, Kim W, Comeau DC et al (2024) Opportunities and challenges for ChatGPT and large language models in biomedicine and health. Brief Bioinform 25(1):493
133. Clark K, Luong M-T, Le QV, Manning CD (2020) Electra: pre-training text encoders as discriminators rather than generators. arXiv:2003.10555
134. Brock A (2018) Large scale GAN training for high fidelity natural image synthesis. arXiv:1809.11096
135. Le Scao T, Fan A, Akiki C, Pavlick E, Ilic S, Hesslow D, Castagné R, Luccioni AS, Yvon F, Galle M (2023) Bloom: a 176b-parameter open-access multilingual language model. arXiv:2301.06001
136. Thoppilan R, De Freitas D, Hall J, Shazeer N, Kulshreshtha A, Cheng H-T, Jin A, Bos T, Baker L, Du Y, et al (2022) Lamda: language models for dialog applications. arXiv:2201.08239
137. Lester B, Al-Rfou R, Constant N (2021) The power of scale for parameter-efficient prompt tuning. arXiv:2104.08691
138. Bertolotti F, Cazzola W. By tying embeddings you are assuming the distributional hypothesis. In: Forty-first international conference on machine learning
139. Liu P, Yuan W, Fu J, Jiang Z, Hayashi H, Neubig G (2023) Pre-train, prompt, and predict: a systematic survey of prompting methods in natural language processing. ACM Comput Surv 55(9):1–35
140. Shaw P, Uszkoreit J, Vaswani A (2018) Self-attention with relative position representations. arXiv:1803.02155
141. Ansar W, Goswami S, Chakrabarti A, Chakraborty B (2023) A novel selective learning based transformer encoder architecture with enhanced word representation. Appl Intell 53(8):9424–9443
142. Dar G, Geva M, Gupta A, Berant J (2022) Analyzing transformers in embedding space. arXiv:2209.02535
143. Hazarika D, Namazifar M, Hakkani-Tür D (2022) Attention biasing and context augmentation for zero-shot control of encoder–decoder transformers for natural language generation. In: Proceedings of the AAAI conference on artificial intelligence, vol 36, pp 10738–10748
144. Lu J, Yao J, Zhang J, Zhu X, Xu H, Gao W, Xu C, Xiang T, Zhang L (2021) Soft: Softmax-free transformer with linear complexity. Adv Neural Inf Process Syst 34:21297–21309
145. Freitag M, Al-Onaizan Y (2017) Beam search strategies for neural machine translation. arXiv:1702.01806
146. Hu EJ, Shen Y, Wallis P, Allen-Zhu Z, Li Y, Wang S, Wang L, Chen W (2021) Lora: low-rank adaptation of large language models. arXiv:2106.09685
147. Ma S, Wang H, Ma L, Wang L, Wang W, Huang S, Dong L, Wang R, Xue J, Wei F (2024) The era of 1-bit LLMs: all large language models are in 1.58 bits. arXiv:2402.17764
148. Frantar E, Ashkboos S, Hoefler T, Alistarh D (2022) OPTQ: accurate quantization for generative pre-trained transformers. In: The 11th international conference on learning representations
149. Tseng A, Chee J, Sun Q, Kuleshov V, De Sa C (2024) Quip#: Even better LLM quantization with hadamard incoherence and lattice codebooks. arXiv:2402.04396
150. Lin J, Tang J, Tang H, Yang S, Chen W-M, Wang W-C, Xiao G, Dang X, Gan C, Han S (2024) AWQ: activation-aware weight quantization for on-device LLM compression and acceleration. Proc Mach Learn Syst 6:87–100
151. Wang H, Ma S, Dong L, Huang S, Wang H, Ma L, Yang F, Wang R, Wu Y, Wei F (2023) Bitnet: scaling 1-bit transformers for large language models. arXiv:2310.11453
152. Wang Y, Mishra S, Alipoormolabashi P, Kordi Y, Mirzaei A, Arunkumar A, Ashok A, Dhanasekaran AS, Naik A, Stap D, et al (2022) Super-naturalinstructions: generalization via declarative instructions on 1600+ NLP tasks. arXiv:2204.07705
153. Bai Y, Jones A, Ndousse K, Askell A, Chen A, DasSarma N, Drain D, Fort S, Ganguli D, Henighan T, et al (2022) Training a helpful and harmless assistant with reinforcement learning from human feedback. arXiv:2204.05862
154. Xiao G, Lin J, Seznec M, Wu H, Demouth J, Han S (2023) Smoothquant: accurate and efficient post-training quantization for large language models. In: International conference on machine learning, pp 38087–38099. PMLR
155. Wortsman M, Dettmers T, Zettlemoyer L, Morcos A, Farhadi A, Schmidt L (2023) Stable and low-precision training for large-scale vision-language models. Adv Neural Inf Process Syst 36:10271–10298
156. Basit SA, Qureshi R, Musleh S, Guler R, Rahman MS, Biswas KH, Alam T (2023) COVID-19Base v3: update of the knowledgebase for drugs and biomedical entities linked to COVID-19. Front Public Health 11:1125917
157. Kitamura FC (2023) ChatGPT is shaping the future of medical writing but still requires human judgment. Radiology 307(2):230171

158. Sallam M (2023) Chatgpt utility in healthcare education, research, and practice: systematic review on the promising perspectives and valid concerns. Healthcare 11(6):887
159. Cascella M, Montomoli J, Bellini V, Bignami E (2023) Evaluating the feasibility of ChatGPT in healthcare: an analysis of multiple clinical and research scenarios. J Med Syst 47(1):33
160. Singhal K, Azizi S, Tu T, Mahdavi SS, Wei J, Chung HW, Scales N, Tanwani A, Cole-Lewis H, Pfohl S, Payne P, Seneviratne M, Gamble P, Kelly C, Scharli N, Chowdhery A, Mansfield P, Arcas B, Webster D, Corrado GS, Matias Y, Chou K, Gottweis J, Tomasev N, Liu Y, Rajkomar A, Barral J, Semturs C, Karthikesalingam A, Natarajan V (2023) Large language models encode clinical knowledge. Nature 620(7972):172–180
161. Karn SK, Ghosh R, Farri O, et al (2023) shs-nlp at radsum23: domain-adaptive pre-training of instruction-tuned LLMs for radiology report impression generation. arXiv:2306.03264
162. Rao A, Kim J, Kamineni M, Pang M, Lie W, Succi MD (2023) Evaluating ChatGPT as an adjunct for radiologic decision-making. MedRxiv, 2023–02
163. Duong D, Solomon BD (2024) Analysis of large-language model versus human performance for genetics questions. Eur J Hum Genet 32(4):466–468
164. Fijačko N, Gosak L, Štiglic G, Picard CT, Douma MJ (2023) Can ChatGPT pass the life support exams without entering the American heart association course? Resuscitation 185:109732
165. Majumder S, Deb Barma MK, Saha A (2024) Arp spoofing detection using machine learning classifiers: an experimental study. Knowl Inf Syst. https://doi.org/10.1007/s10115-024-02219-y
166. Romano MF, Shih LC, Paschalidis IC, Au R, Kolachalama VB (2023) Large language models in neurology research and future practice. Neurology 101(23):1058–1067
167. Haque MR, Rubya S (2023) An overview of chatbot-based mobile mental health apps: insights from app description and user reviews. JMIR mHealth and uHealth 11(1):44838
168. Jungmann SM, Klan T, Kuhn S, Jungmann F (2019) Accuracy of a chatbot (ADA) in the diagnosis of mental disorders: comparative case study with lay and expert users. JMIR Formative Res 3(4):13863
169. Magalhaes Azevedo D, Kieffer S (2021) User reception of AI-enabled mHealth apps: the case of Babylon health
170. Malik P, Pathania M, Rathaur VK et al (2019) Overview of artificial intelligence in medicine. J Family Med Primary Care 8(7):2328–2331
171. Thawkar O, Shaker A, Mullappilly SS, Cholakkal H, Anwer RM, Khan S, Laaksonen J, Khan FS (2023) Xraygpt: chest radiographs summarization using medical vision-language models. arXiv:2306.07971
172. Ma J, He Y, Li F, Han L, You C, Wang B (2024) Segment anything in medical images. Nat Commun 15(1):654
173. Dowling M, Lucey B (2023) ChatGPT for (finance) research: the Bananarama conjecture. Finance Res Lett 53:103662
174. Zaremba A, Demir E (2023) ChatGPT: unlocking the future of NLP in finance. Modern Finance 1(1):93–98
175. Lopez-Lira A, Tang Y (2023) Can ChatGPT forecast stock price movements? Return predictability and large language models. arXiv:2304.07619
176. Yang Y, Uy MCS, Huang A (2020) Finbert: a pretrained language model for financial communications. arXiv:2006.08097
177. Peskoff D, Stewart BM (2023) Credible without credit: domain experts assess generative language models. In: Proceedings of the 61st annual meeting of the association for computational linguistics (volume 2: short papers), pp 427–438
178. Hansen KB (2020) The virtue of simplicity: on machine learning models in algorithmic trading. Big Data Soc 7(1):2053951720926558
179. Yang H, Liu X-Y, Wang CD (2023) Fingpt: open-source financial large language models. arXiv:2306.06031
180. Pavlik JV (2023) Collaborating with ChatGPT: considering the implications of generative artificial intelligence for journalism and media education. J Mass Commun Educ 78(1):84–93
181. Hauptman AI, Mallick R, Flathmann C, McNeese NJ (2024) Human factors considerations for the context-aware design of adaptive autonomous teammates. Ergonomics. https://doi.org/10.1080/00140139.2024.2380341
182. Kirchenbauer J, Geiping J, Wen Y, Katz J, Miers I, Goldstein T (2023) A watermark for large language models. In: International conference on machine learning, pp 17061–17084. PMLR
183. Wang Z (2023) MediaGPT: a large language model target Chinese media. arXiv:2307.10930
184. Pérez JM, Furman DA, Alemany LA, Luque F (2021) RoBERTuito: a pre-trained language model for social media text in Spanish. arXiv:2111.09453
185. Park PS, Schoenegger P, Zhu C (2024) Diminished diversity-of-thought in a standard large language model. Behav Res Methods 56:5754–5770

186. Mayahi S, Vidrih M (2022) The impact of generative AI on the future of visual content marketing. arXiv:2211.12660
187. Kim J, Xu K, Merrill K Jr (2022) Man vs. machine: human responses to an AI newscaster and the role of social presence. Soc Sci J. https://doi.org/10.1080/03623319.2022.2027163
188. Kushwaha AK, Kar AK (2020) Language model-driven chatbot for business to address marketing and selection of products. In: Re-imagining diffusion and adoption of information technology and systems: a continuing conversation: IFIP WG 8.6 international conference on transfer and diffusion of IT, TDIT 2020, Tiruchirappalli, India, December 18–19, 2020, Proceedings, Part I, pp 16–28. Springer
189. Rivas P, Zhao L (2023) Marketing with ChatGPT: navigating the ethical terrain of GPT-based chatbot technology. AI 4(2):375–384
190. Tembhurne JV, Lakhotia K, Agrawal A (2024) Twitter sentiment analysis using ensemble of multi-channel model based on machine learning and deep learning techniques. Knowl Inf Syst. https://doi.org/10.1007/s10115-024-02256-7
191. Verma S, Sharma R, Deb S, Maitra D (2021) Artificial intelligence in marketing: systematic review and future research direction. Int J Inf Manag Data Insights 1(1):100002
192. Sun GH, Hoelscher SH (2023) The ChatGPT storm and what faculty can do. Nurse Educator 48(3):119–124
193. Stone M, Aravopoulou E, Ekinci Y, Evans G, Hobbs M, Labib A, Laughlin P, Machtynger J, Machtynger L (2020) Artificial intelligence (AI) in strategic marketing decision-making: a research agenda. The Bottom Line 33(2):183–200
194. Jarek K, Mazurek G (2019) Marketing and artificial intelligence. Central Eur Bus Rev 8(2):46–55
195. Subagja AD, Ausat AM, Sari AR, Wanof M, Suherlan S (2023) Improving customer service quality in MSMEs through the use of ChatGPT. Jurnal Minfo Polgan 12(1):380–386
196. Howell K, Christian G, Fomitchov P, Kehat G, Marzulla J, Rolston L, Tredup J, Zimmerman I, Selfridge E, Bradley J (2023) The economic trade-offs of large language models: a case study. arXiv:2306.07402
197. Allen DW, Berg C, Ilyushina N, Potts J (2023) Large language models reduce agency costs. Available at SSRN 4437679
198. Olujimi PA, Ade-Ibijola A (2023) NLP techniques for automating responses to customer queries: a systematic review. Discover Artif Intell 3(1):20
199. Makridakis S, Petropoulos F, Kang Y (2023) Large language models: their success and impact. Forecasting 5(3):536–549
200. Glaese A, McAleese N, Trebacz M, Aslanides J, Firoiu V, Ewalds T, Rauh M, Weidinger L, Chadwick M, Thacker P, Campbell-Gillingham L, Uesato J, Huang PS, Comanescu R, Yang F, See A, Dathathri S, Greig R, Chen C, Fritz D, Sanchez Elias J, Green R, Mokrá S, Fernando N, Wu B, Foley R, Young S, Gabriel I, Isaac W, Mellor J, Hassabis D, Kavukcuoglu K, Hendricks LA, Irving G (2022) Improving alignment of dialogue agents via targeted human judgements. arXiv:2209.14375
201. ChatGPT (2022) OpenAI: optimizing language models for dialogue. OpenAI
202. Schulman J, Zoph B, Kim C, Hilton J, Menick J, Weng J, Ceron Uribe JF, Fedus L, Metz L, Pokorny M (2022) ChatGPT: optimizing language models for dialogue. OpenAI Blog 2(4):1–7
203. Köpf A, Kilcher Y, Rütte D, Anagnostidis S, Tam ZR, Stevens K, Barhoum A, Nguyen D, Stanley O, Nagyfi R (2024) Openassistant conversations-democratizing large language model alignment. In: Advances in neural information processing systems, vol 36
204. Chen M, Papangelis A, Tao C, Kim S, Rosenbaum A, Liu Y, Yu Z, Hakkani-Tur D (2023) Places: Prompting language models for social conversation synthesis. arXiv:2302.03269
205. Kocoń J, Cichecki I, Kaszyca O, Kochanek M, Szydło D, Baran J, Bielaniewicz J, Gruza M, Janz A, Kanclerz K et al (2023) ChatGPT: Jack of all trades, master of none. Inf Fus 99:101861
206. Bubeck S, Chandrasekaran V, Eldan R, Gehrke J, Horvitz E, Kamar E, Lee P, Lee Y, Li Y, Lundberg S, Nori H, Palangi H, Ribeiro M, Zhang Y (2023) Sparks of artificial general intelligence: early experiments with GPT-4. arXiv:2303.12712
207. Yang J, Jin H, Tang R, Han X, Feng Q, Jiang H, Zhong S, Yin B, Hu X (2024) Harnessing the power of llms in practice: A survey on ChatGPT and beyond. ACM Trans Knowl Discov Data 18(6):1–32
208. Akram A (2023) An empirical study of AI generated text detection tools. arXiv:2310.01423
209. OpenAI T (2022) ChatGPT: optimizing language models for dialogue. OpenAI
210. Christian J (2023) CNET secretly used AI on articles that didn't disclose that fact, staff say. Futurism
211. Mitchell E, Lee Y, Khazatsky A, Manning CD, Finn C (2023) Detectgpt: zero-shot machine-generated text detection using probability curvature. In: International conference on machine learning, pp 24950–24962. PMLR, London, UK
212. Gehrmann S, Strobelt H, Rush AM (2019) Gltr: statistical detection and visualization of generated text. arXiv:1906.04043

213. Atallah MJ, Raskin V, Crogan M, Hempelmann C, Kerschbaum F, Mohamed D, Naik S (2001) Natural language watermarking: design, analysis, and a proof-of-concept implementation. In: Information hiding: 4th international workshop. IH 2001 Pittsburgh, PA, USA, April 25–27, 2001 Proceedings 4. Springer, Berlin, pp 185–200
214. Kumar A, Levine A, Goldstein T, Feizi S (2022) Certifying model accuracy under distribution shifts. arXiv:2201.12440
215. Quidwai A, Li C, Dube P (2023) Beyond black box AI generated plagiarism detection: from sentence to document level. In: Kochmar E, Burstein J, Horbach A, Laarmann-Quante R, Madnani N, Tack A, Yaneva V, Yuan Z, Zesch T (eds), Proceedings of the 18th workshop on innovative use of NLP for building educational applications (BEA 2023), pp 727–735. Association for computational linguistics, Toronto, Canada. https://doi.org/10.18653/v1/2023.bea-1.58 . https://aclanthology.org/2023.bea-1.58
216. Krishna K, Song Y, Karpinska M, Wieting J, Iyyer M (2024) Paraphrasing evades detectors of AI-generated text, but retrieval is an effective defense. In: Advances in neural information processing systems, vol 36
217. Wu J, Yang S, Zhan R, Yuan Y, Wong DF, Chao LS (2023) A survey on LLM-generated text detection: necessity, methods, and future directions. arXiv:2310.14724
218. Collobert R, Weston J, Bottou L, Karlen M, Kavukcuoglu K, Kuksa P (2011) Natural language processing (almost) from scratch. J Mach Learn Res 12:2493–2537
219. Blanchard A, Taddeo M (2023) The ethics of artificial intelligence for intelligence analysis: a review of the key challenges with recommendations. Digit Soc 2(1):12
220. Abdullah T, Ahmet A (2022) Deep learning in sentiment analysis: recent architectures. ACM Comput Surv 55(8):1–37
221. Brundage M, Avin S, Clark J, Toner H, Eckersley P, Garfinkel B, Dafoe A, Scharre P, Zeitzoff T, Filar B, et al (2018) The malicious use of artificial intelligence: Forecasting, prevention, and mitigation. arXiv:1802.07228
222. Das A, Verma RM (2020) Can machines tell stories? A comparative study of deep neural language models and metrics. IEEE Access 8:181258–181292
223. Ippolito D, Duckworth D, Callison-Burch C, Eck D (2019) Automatic detection of generated text is easiest when humans are fooled. arXiv:1911.00650
224. Fröhling L, Zubiaga A (2021) Feature-based detection of automated language models: tackling GPT-2, GPT-3 and Grover. PeerJ Comput Sci 7:443
225. Zellers R, Holtzman A, Rashkin H, Bisk Y, Farhadi A, Roesner F, Choi Y (2019) Defending against neural fake news. In: Advances in neural information processing systems, vol 32
226. Singh M, Kumar R, Chana I (2021) Machine translation systems for Indian languages: review of modelling techniques, challenges, open issues and future research directions. Arch Comput Methods Eng 28(4):2165–2193
227. Meetei LS, Singh TD, Bandyopadhyay S (2024) An empirical study of a novel multimodal dataset for low-resource machine translation. Knowl Inf Syst 66(11):7031–7055
228. Sinha S, Jha GN (2022) An overview of Indian language datasets used for text summarization. In: ICT with intelligent applications: proceedings of ICTIS 2022, Volume 1, pp 693–703. Springer, Berlin
229. Singh M, Kumar R, Chana I (2021) Improving neural machine translation for low-resource Indian languages using rule-based feature extraction. Neural Comput Appl 33(4):1103–1122
230. McKenna N, Li T, Cheng L, Hosseini MJ, Johnson M, Steedman M (2023) Sources of hallucination by large language models on inference tasks. arXiv:2305.14552
231. Blodgett SL, Barocas S, Daumé III H, Wallach H (2020) Language (technology) is power: a critical survey of 'bias' in NLP. arXiv:2005.14050
232. Gehman S, Gururangan S, Sap M, Choi Y, Smith NA (2020) RealToxicityPrompts: Evaluating neural toxic degeneration in language models. In: Cohn T, He Y, Liu Y (eds), Findings of the association for computational linguistics: EMNLP 2020, pp. 3356–3369. Association for computational linguistics, Online. https://doi.org/10.18653/v1/2020.findings-emnlp.301. https://aclanthology.org/2020.findings-emnlp.301
233. Carlini N, Tramer F, Wallace E, Jagielski M, Herbert-Voss A, Lee K, Roberts A, Brown T, Song D, Erlingsson U (2021) Extracting training data from large language models. In: 30th USENIX security symposium (USENIX Security 21), pp 2633–2650
234. Kaur R, Bhatia M, Kumar A (2024) Am I Hurt?: evaluating psychological pain detection in Hindi text using transformer-based models. ACM Trans Asian Low-Resource Lang Inf Process. https://doi.org/10.1145/3650206
235. Zhou X, Zhang Y, Cui L, Huang D (2020) Evaluating commonsense in pre-trained language models. In: Proceedings of the AAAI conference on artificial intelligence, vol 34, pp 9733–9740

236. Bukkuri A, Pienta KJ, Austin RH, Hammarlund EU, Amend SR, Brown JS (2022) Stochastic models of mendelian and reverse transcriptional inheritance in state-structured cancer populations. Sci Rep 12(1):13079
237. Frankle J, Carbin M (2018) The lottery ticket hypothesis: finding sparse, trainable neural networks. arXiv:1803.03635

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

Yadagiri Annepaka is a Ph.D. student in the Department of Computer Science and Engineering, National Institute of Technology Silchar, India. His research focuses on natural language processing, deep learning and large language models.

Dr. Partha Pakray is an Associate Professor in the Department of Computer Science and Engineering at the National Institute of Technology (NIT). He worked as a Postdoctoral Fellow in the natural language processing (NLP) domain at Masaryk University, Czech Republic. He was also a Postdoctoral Fellow at the Norwegian University of Science and Technology from 2013 to 2014. He worked as a Senior Research Engineer at Jadavpur University on the "Development of Cross-Lingual Information Access (CLIA) Phase II" project under the Ministry of Communications and Information Technology, Government of India (2007 - 2013). His international collaborations include the CONACYT-DST India project titled "Answer Validation through Textual Entailment" with Alexander Gelbukh from the National Polytechnic Institute, Mexico, and Professor Sivaji Bandyopadhyay from Jadavpur University, India. He visited Mexico in 2011 and 2013 for this project. In 2012, Partha Pakray completed a six-month research internship at Xerox Research Centre Europe (XRCE) in Grenoble, France. He got the workshop project funding on "Quantum Machine Learning for Natural Language Processing Applications" from MeitY Quantum Computing Applications Lab (QCAL). His research expertise focuses on Quantum Machine Learning for Natural Language Processing Application. He has published more than 150 papers in reputed journals, conferences and workshops proceedings, and 5 patents. He received 8 research funded projects from various Government of India funding agencies like DST, SERB, CEFIPRA, DAAD, ASEAN, MeitY Quantum Computing Applications Lab (QCAL). He is very active in several international conferences organized by local and international associations, institutions or universities and is also an editorial board member of Natural Language Engineering (Cambridge Press) journal.